

Kirei

Commercial Readiness & Polish Audit

Scope Full product — UI, functionality, testing, docs, security, performance

Method 15 agents: 6 domain specialists → adversarial verification → synthesis

Findings 46 confirmed · 3 refuted in verification

Market 8 competitors researched; design analysis grounded in the live token system

Generated 18 July 2026

Kirei — Commercial Readiness Report

1. Verdict

You have built roughly three products and finished none of them commercially. The one that matters — field services — has a real, dependent user (a roofer with \$17k outstanding, \$327k open quotes, 98 jobs) and is genuinely competitive on features. But you cannot charge anyone for Kirei (no subscription billing exists anywhere in the codebase), your AI differentiator returns 401 in production, and the audit found a live cross-tenant data leak: `sms_conversations`, `sms_messages` and `report_sessions` have **no RLS at all** and the SMS tables are published to Realtime, so any authenticated user of any tenant can read every business's customer phone numbers and message bodies (`supabase/migrations/20260413000002_sms.sql:3-37`).

The single most important thing to do next: ship the RLS migration and fail-close the cron auth this week, then build subscription billing. Everything else — features, design, the SEO platform — is downstream of being able to take money safely.

2. Ship blockers

Ordered. These must be fixed before a stranger pays you.

B1. Three tables have no RLS; SMS is streamed cross-tenant over Realtime

Where: `supabase/migrations/20260413000002_sms.sql:3-37` (creates `sms_conversations` + `sms_messages`, no `ENABLE ROW LEVEL SECURITY`, then `ALTER PUBLICATION supabase_realtime ADD TABLE` on both at lines 36-37); `supabase/migrations/20260413000003_report_sessions.sql:4-17`. No later migration adds policies. `src/components/messages/messages-client.tsx` subscribes to `sms-conversations` with event `*` and **no filter**. **Why it blocks:** Signup is public. Any authenticated user reads every tenant's customer names, E.164 phone numbers and message text with the anon key, and Realtime pushes other tenants' rows into their browser live. `report_sessions` leaks customer ids and property addresses the same way. This is a reportable breach in most jurisdictions and an instant loss in any security review. **Effort:** hours. One migration: enable RLS on all three, standard owner-UNION-active-member policy, `AND NOT public.is_business_worker(business_id)` on the SMS pair, deny-all on `report_sessions` (only written by the admin client). Add `filter: business_id=eq.<id>` to the Realtime channel.

B2. Twelve cron routes fail OPEN — including the one that charges saved cards

Where: `src/app/api/cron/recurring-invoices/route.ts:22` — `if (process.env.CRON_SECRET && auth !== ...) return 401`. Same shape at `daily-digest:41`, `invoice-reminders:33`, `process-scheduled-sends:205`, `quote-followup:33`, `recurring-jobs:30`, `reminders:78`, `seo-audits:17`, `seo-gsc-sync:23`, `seo-jobs:20`, `seo-reports:16`, `workorder-complete:29`. `/api/cron/` is unconditionally public in `src/lib/supabase/middleware.ts:48,68`. Only `content-jobs/route.ts:27` and `email-leads/route.ts:396-398` fail closed. **Why it blocks:** If the variable is ever unset, renamed or missing in an environment (preview environments point at the **production** database), an unauthenticated GET generates invoices and off-session charges customers' saved cards via `src/lib/recurring/generate-invoice.ts:77-80` — on a live Stripe

integration, up to 6 catch-up cycles per schedule per call. `invoice-reminders` and `quote-followup` can be looped to spam every tenant's customers from a verified domain. **Effort:** minutes. One `requireCron(req)` helper that 500s when the secret is unset and 401s on mismatch, used in all fourteen routes including `booking-reminders:19-20`. Then verify `CRON_SECRET` exists in every Vercel environment and redeploy (env changes don't apply to a running deployment).

B3. There is no way to charge for Kirei

Where: Nowhere. No subscription, plan, trial or billing-period code exists in `src/lib/actions` or the settings pages. Stripe Connect lets *your customers* take money; there is no mechanism for *you* to be paid. **Why it blocks:** Literally. You cannot sell the product today. **Effort:** days. Stripe Billing (not Connect) on the platform account, a `plan / trial_ends_at` on `businesses`, a paywall gate in `(dashboard)/layout.tsx`, and a billing page. Keep it dumb: three prices, one webhook, no prororation logic in v1.

B4. `addPayment` discards the insert error — a failed payment reports success

Where: `src/lib/actions/invoices.ts:351` — bare `await tbl(supabase, "payments").insert({...})`, no destructure, no throw. Execution falls through to the recompute at `:357-372`, which re-reads the payments table and writes the pre-existing `amount_paid` back over itself. The updates at `:372` and `:417` are also unchecked. No validation of `payment.amount` anywhere in the function. **Why it blocks:** On any insert failure (RLS denial, CHECK violation, bad date) the UI reports success and revalidates, and the user believes money was recorded when nothing was written. Same shape as the `.or()` incident in your own Known-traps list. Reachable by editor-role users and the MCP/assistant surface. **Effort:** minutes. `const { error } = ...; if (error) throw error;` on all three writes, plus `if (!Number.isFinite(amount) || amount <= 0) throw`.

B5. Marking an invoice paid, then recording a payment, silently un-pays it

Where: `src/lib/actions/invoices.ts:96-99` (`updateInvoice` sets `amount_paid = cur.total` on `status→paid`, writes no payments row) vs `:357-372` (`addPayment` recomputes `amount_paid` purely from the payments table and overwrites). The parent-rollup branch at `:409-415` correctly refuses to fight `cancelled / draft`; the invoice's own status write at `:370` has no such guard. **Why it blocks:** Mark a \$10,000 invoice paid from the dropdown, then record any later payment: `amount_paid` drops to that payment's value and status flips to `partial`. A settled invoice reads as outstanding on the dashboard and becomes a target for the overdue cron — which then emails the customer chasing money they already paid. Note the recompute itself is deliberate (it fixes progress-invoice rollup); preserve that behaviour. **Effort:** hours. Make payments the single source of truth: have the mark-paid branch insert a balancing payments row ("Manual — marked paid") instead of writing the column directly.

B6. Every business defaults to GBP; nothing checks it against the Stripe account

Where: `supabase/migrations/001_initial_schema.sql:20` — `currency text not null default 'GBP'`. Read at `src/app/api/stripe/checkout/route.ts:68` (line items `:83`, fee `:70`) and `src/lib/stripe-charge.ts:59`. `handleAccountUpdated` (`src/app/api/stripe/webhook/route.ts:54-64`) receives the Stripe Account and persists only `country` — never `default_currency`. Signup never sets currency; the only change path is `src/components/settings/settings-client.tsx:353-360`. **Why it blocks:** Wrong default for an Australian product, and no guard anywhere against charging a currency the connected account doesn't settle in.

The live payment path has never been exercised by a real customer, so the first one is the test. **Effort:** hours. Persist `account.default_currency`, treat it as source of truth for charge currency, hard-refuse on mismatch in both charge paths, change the schema default, force the choice at signup.

B7. `npm test` writes to the production database

Where: `src/lib/booking/__tests__/booking-db.test.ts:29` selects `businesses ... limit(1)` and inserts a `VITEST_TMP` booking resource into whichever tenant is first (`:31-33`), then 8 concurrent appointments (`:42-63`). Gate is just "env vars present" (`:17`); `setup.ts:2-4` loads `.env.local`, which holds production keys. Cleanup is `afterAll` only, so an interrupt orphans the rows. CI never runs it (no secrets), so it only ever runs against prod. **Why it blocks:** A phantom resource and year-2099 appointments in a paying customer's UI. Also means the documented verification command is unsafe. **Effort:** hours. Gate behind `RUN_DB_TESTS=1` (the pattern `src/lib/mcp/__tests__/live-api.test.ts:23` already uses), create a throwaway business in `beforeAll` rather than `limit(1)`, wrap cleanup in `try/finally`.

B8. `ANTHROPIC_API_KEY` returns 401; `SENTRY_DSN` unset

Why it blocks: Every AI feature — the assistant, quoting agent, SEO pipeline, Content Studio — is dead in production. A prospect clicking "Assistant" in a trial sees an abandoned product, and the AI is your only defensible differentiator. Separately, `src/instrumentation.ts` is inert without a DSN, so with paying customers you learn about outages from angry phone calls. **Effort:** minutes each. Top up the account, set the DSN.

B9. Worker isolation stops at six tables

Where: `supabase/migrations/20260430000001_worker_role_and_isolation.sql` applies `is_business_worker` to exactly `customers:76`, `products:97`, `invoices:118`, `quotes:139`, `payments:160`, `reports:181`. Not applied to: `sites` (`20260418000001:51`), `contacts` (`20260428000002:71` , the live policy), `billing_profiles` (`20260418000001:148`), `leads` (`20260412000001:42`), `customer_notes` (`20260403000001:51`). **Why it blocks:** A worker — the role you document as hard-isolated — can SELECT/INSERT/UPDATE/DELETE every serviced address, every contact, the entire sales pipeline and all customer notes, via the mobile Supabase client or raw PostgREST. Half the guarantee isn't real, and it's invisible from the UI. This is the hardest question in a prospect's security review. **Effort:** hours. One migration replacing those five policies with `AND NOT public.is_business_worker(business_id)`.

B10. OAuth consent mints a permanent admin-scope key for *any* member, including workers

Where: `src/app/api/oauth/authorize/route.ts` — `listBusinesses` filters on `user_id` + `status='active'` only; `role` is never read. `src/app/api/oauth/token/route.ts` inserts a `business_api_keys` row with `scopes: ["admin"]` and `no expires_at`. Contrast `src/lib/actions/api-keys.ts` (gates on `canManageSettings`) and `src/lib/assistant/scopes.ts` (returns `null` for worker precisely because the admin client bypasses RLS). **Why it blocks:** A worker completing the claude.ai connector flow gets a never-expiring wildcard key with read/write on every customer, invoice, quote and payment through ~199 MCP tools. Insider-only, which is why it's below B1, but it's the exact escalation your scope layer exists to prevent. **Effort:** hours. Join `role` in `listBusinesses` and drop worker/viewer; replace `["admin"]` with `assistantScopesForRole(role)`; set `expires_at`.

3. Polish customers will notice

Ordered by visible impact per hour.

#	Fix	Where	Effort
P1	Six one-click deletes with no confirmation. A misclick on a 16px trash icon — sitting next to the receipt-view icon — permanently destroys a financial record. <code>assets.ts:72-77</code> is a hard row delete. <code>leads-client.tsx:445-459</code> already has the AlertDialog pattern to copy.	<code>expenses-view.tsx:86-91,136</code> ; <code>assets-view.tsx:57,107</code> ; <code>seo-reports.tsx:40,87</code> ; <code>seo-connections.tsx:92,297</code> ; <code>content-brands-view.tsx:74,199</code> ; <code>content-campaigns-view.tsx:149</code>	hours
P2	No <code>error.tsx</code> or <code>not-found.tsx</code> anywhere. 89 pages, 47 <code>notFound()</code> call sites, zero boundaries. Every 404 and every server-component throw renders Next's bare black-and-white page with no sidebar and no way back — and with no Sentry you never learn it happened.	Add <code>(dashboard)/error.tsx</code> , <code>(dashboard)/not-found.tsx</code> , root <code>global-error.tsx</code> + <code>not-found.tsx</code> . Compose from PageHeader + EmptyState.	hours
P3	The overdue-payment email contains no way to pay. <code>invoice-reminders/route.ts:99-126</code> builds inline HTML with zero links — no portal link, no pay link. The imported <code>btn</code> helper at <code>:14</code> is never used. Your highest-intent email offers the customer nothing to click.	<code>src/app/api/cron/invoice-reminders/route.ts</code> — reuse the token helper at <code>generate-invoice.ts:131-144</code>	hours
P4	Cron emails go out from Crown Roofers' address, for every tenant. <code>reminders/route.ts:17</code> defaults to <code>Crown Roofers <noreply@crownroofers.com.au></code> and <code>:176</code> hardcodes <code>replyTo: info@crownroofers.com.au</code> . <code>invoice-reminders:19</code> , <code>quote-followup:19</code> , <code>workorder-complete:19</code> use the global <code>RESEND_FROM_EMAIL</code> (production = the Crown Roofers address). None set <code>replyTo</code> , none call <code>getResolvedEmailTemplate</code> — so the per-business email templates you built in June are silently ignored on every one of these paths. <code>process-scheduled-sends:119,184</code> does it correctly; copy that.	4 cron routes	hours
P5	<code>/api/cron/reminders</code> is hardcoded to one business UUID. <code>route.ts:14</code> pins <code>BUSINESS_ID</code> , <code>:68</code> is the only <code>business_id</code> filter in the file, one Telegram chat at <code>:16</code> . Runs daily. Every other tenant silently gets nothing.	Fan out over <code>business_agent_installs</code> like <code>invoice-reminders:43-47</code> , or delete the route and its <code>vercel.json</code> entry	hours
P6	Plugin list pages have no search. Zero search inputs in expenses, assets, inventory, timesheets, contracts. <code>CleanupButton</code> is imported in only 8 files against CLAUDE.md's claim that it's on "every list page". A prospect comparing to ServiceM8 will try to search the first list they open.	Port the <code>leads-client.tsx:308-317</code> pattern	days

#	Fix	Where	Effort
P7	In-app help covers 14 of 32 modules and never mentions payments. <code>help-client.tsx:16-130</code> has no section on Stripe, card payments, deposits, saved cards, contracts, e-signature, forms or bookings. A customer cannot discover from inside the product that Kirei takes card payments — plausibly a contributing cause of zero payments ever.	<code>src/components/help/help-client.tsx</code> — lead with "Getting paid"	days
P8	PageHeader truncates the title on every page, against your own documented break-words rule. <code>page-header.tsx:30</code> , one-word fix.	<code>truncate</code> → <code>break-words</code>	minutes
P9	Refunds, disputes and failed bank payments are unhandled. The webhook switch (<code>webhook/route.ts:31-45</code>) covers 4 events; no reversal logic exists anywhere in <code>src/</code> . A refund leaves the invoice reading paid and still counting as revenue. Not yet damaging (0 payments), but it's the second real customer interaction that breaks it.	<code>charge.refunded</code> , <code>charge.dispute.created</code> , <code>checkout.session.async_payment_failed</code>	days
P10	.ch-table rows show a pointer cursor and hover highlight but aren't clickable (<code>globals.css:446-450</code> ; rows in <code>inventory/assets/expenses/timesheets</code> have no <code>onClick</code>). One-line CSS: move to an opt-in <code>.ch-table.is-clickable</code> .	<code>globals.css</code>	minutes

4. Design direction

The token layer in `globals.css:134-179` is genuinely good — warm off-white canvas, deep teal, the tracking corrections at `:118-125` show real craft. And `lead-card.tsx:3-11` contains a written design rationale ("the old card filled its whole surface with one of five stage colours... a full board was five columns of saturated blocks and nothing stood out") that arrives at exactly the right answer. **The work is not new design. It is applying that one paragraph of reasoning to the other twenty screens.**

Two are outright bugs, not taste:

1. **.ch-pill has no dark-mode variant.** Backgrounds are hardcoded literals (`hsl(150 50% 95%)` etc., `globals.css:474-493`) with no `.dark` override. In dark mode they render as near-white blocks — on `worker-dashboard.tsx`, `messages-client.tsx` and `briefing-bell.tsx`. The worker dashboard is the crew screen used on a phone outdoors, which is precisely where dark mode gets used. `.ch-stat-delta.up/.down` (`:412-413`) has the same defect. **Fix:** convert to custom properties, override in the existing dark block at `:158-179`.

2. **Every gradient flashes light on load in dark mode.** `useIsDark()` is duplicated verbatim in `gradient-tile.tsx:25-37`, `stat-tile.tsx:25-37` and `avatar.tsx:26-38`, each initialising to `false` and correcting in `useEffect`. SSR and first paint always emit the light palette. On the dashboard that's a dozen elements strobing. **The fix is already written and unused:** `gradientVars()` at `gradient-tokens.ts:63-74` emits the CSS custom properties for exactly this. Delete the three hooks, consume `gradientVars()`, add one dark remap rule. Removes three `"use client"` boundaries as a bonus.

Then, in priority order:

3. **Collapse four status vocabularies into one.** `.ch-pill` (`globals.css:455-493`), `KireiPill` (`ui/kirei/pill.tsx:44-51`), leads `STAGES` (`lead-shared.ts:23-29`), and `STATUS_GRADIENT` icon tiles (`invoices-client.tsx:183`). "Paid" is a warm mint pill on /messages, a saturated ALL-CAPS emerald pill on /invoices, and an emerald gradient square on the same row. Make `KireiPill` the single primitive, re-source its tones from the Connected Hub tokens rather than raw Tailwind, keep `.ch-pill`'s leading dot (helps colour-blind users) and its 11.5px sentence case (more legible in sunlight than 10px caps), and rewrite `.ch-pill` as a thin alias.
4. **Desaturate.** `KireiAvatar` hashes names into nine fully-saturated gradients (`avatar.tsx:12`, `gradient-tokens.ts:12-29`) — a 50-row customer list is a rainbow where colour means nothing, which trains the user to ignore colour exactly where you later need it to shout. Reduce to 4-5 teal tints, or go flat muted like Linear and Stripe. Same for the dashboard's four differently-coloured quick actions (`dashboard-client.tsx:150-153`) — four equal actions implying a taxonomy that doesn't exist. A tradie evaluating you against ServiceM8 and Tradify, whose entire claim is calm simplicity, reads visual noise as "app-builder template".
5. **Invert the invoice row hierarchy.** `invoices-client.tsx:170-200`: a 40px saturated gradient tile carrying zero information leads, and the *money* renders at `text-sm font-semibold` — same weight as the customer name. Drop the tile (status is already in the pill), render the total `text-base font-bold tabular-nums` right-aligned. The row is also a table pretending not to be one (`w-24` / `w-28` / `w-32` spans, `divide-y` divs, `flex-wrap` that reflows to ragged heights at tablet width) while seven other views use real `.ch-table`. Convert to `.ch-table` at `md:` and up, keep cards below.
6. `layout.tsx:9` sets `themeColor: "#2563eb"` — bright blue from the abandoned `:root` theme. On an installed PWA and on Android that's the browser chrome and status bar. It is the first colour a customer sees on their phone, and it belongs to a design system you don't use. Set it to the teal or the canvas pair, and delete the dead `:root` blue block at `globals.css:40-102` (which also carries a second, contradictory radius scale).
7. Lower priority: standardise on one KPI treatment (`.ch-stat` vs `StatTile`, and `StatTile:53` hardcodes `rgba(255,255,255,0.55)` — glary on a dark tile); consolidate the two empty-state treatments and make `cta` required (a first-run user staring at eight empty screens with no CTA is your most expensive conversion failure); replace literal px radii in `.ch-*` (`globals.css:382,420`) with the token.
-

5. Competitive position

Where you genuinely win

- **First-party MCP server with full OAuth 2.1.** Nobody else has this. The only comparable thing in the market is an unofficial community wrapper around the Housecall Pro REST API. A tradie adds Kirei as a claude.ai connector and runs their business by talking to Claude.
- **One tool registry serving MCP and the in-app assistant** (`register-tools.ts` takes a `ToolFn`; `collect.ts` reuses it). Every competitor's AI is bolted onto a fixed feature list. Yours is the app.
- **Undo on agent actions** (`src/lib/assistant/undo.ts`) — the answer to the only objection a tradie will raise ("what if it stuffs up my invoices").
- **Native e-signature contracts at zero marginal cost.** Competitors either don't do contracts or make you pay a third party.
- **GHL-class CRM inside a trades product** — lead dedup at the database level, public forms with embeds, onboarding forms. Getting this alongside ServiceM8 costs USD \$97–497/mo from GoHighLevel. It is currently invisible in your marketing.
- **Worker isolation in the database** rather than the UI (once B9 is fixed).

Where you lose

- **No Xero/MYOB/QuickBooks. At all.** I grepped: the only hit in `src/` is a prompt string in `email-leads/route.ts:150`. Every competitor leads with two-way Xero sync. In Australia the bookkeeper has veto and will not accept re-keying invoices. **Nothing else on this list matters until Xero exists.**
- **Android isn't shipped.** The `.aab` is built; the first Play release needs a manual Console upload that hasn't happened. ServiceM8's most-cited weakness in AU reviews is being iOS-only. This is your single clearest wedge against the market leader and it's sitting behind a form upload. Hours of work.
- **No offline capability.** `mobile/` has AsyncStorage and MMKV but no write queue or sync reconciliation. A roofer in a Telstra blackspot who loses a job photo churns that week and tells other roofers.
- **No GPS** (no `expo-location`). Any business with 3+ vehicles asks on the first call.
- **No migration path in.** No importer for ServiceM8, Tradify or a customer CSV. Switching costs are the entire moat in this category.
- **No job-costing page.** `get_job_costing` exists as an MCP tool, but margin-per-job is the specific reason AU/NZ tradies leave Tradify for Fergus. It needs to be a page an owner opens weekly.
- One Supabase project for local/preview/prod. 14 test files against 88k lines.

Positioning

"ServiceM8 for Android, that you can run by talking to it." Australian trades, 2–10 people. Not solo tradies (ServiceM8's free tier owns that and you can't win a price war), not commercial contractors (simPRO owns it, you have no project management), and emphatically not agencies.

Three attackable weaknesses: (1) ServiceM8 has no Android — ship yours; (2) everyone else charges per user, which is the loudest complaint in every review corpus — charge per business; (3) ServiceM8 and Tradify have no CRM — you have one already.

Lead marketing with jobs, quotes, invoices, photos and Android. The AI is the closer, not the opener: a 90-second demo of *"quote the Henderson roof, three days, two guys, send it"*. Tradies don't buy "AI-powered", they buy "I did my quoting in the ute in four minutes."

Pricing

Flat per business, AUD ex GST, unlimited users, no job caps.

Tier	Price	Includes
Solo	A\$79/mo	1–2 users, unlimited jobs/quotes/invoices/photos, portal, Xero, iOS + Android, AI (~300 msg/mo)
Crew	A\$149/mo	Up to 10 users, + leads & public forms, contracts + e-sign, recurring billing & card-on-file, job costing, SMS, unlimited AI
Business	A\$299/mo	Unlimited users, multi-business, API + MCP, priority support, custom branding

Crew is the whole argument. A\$149 for 10 users vs Tradify ~\$520, Fergus ~\$440, Jobber ~A\$530, simPRO ~\$700–1200 plus \$3–10k implementation and a multi-year contract escalating ~8%/yr. You are 3–4x cheaper at team scale and that's a sentence tradies repeat to each other. Below ~4 users the per-user products beat you — accept it, that's why Solo exists.

Solo at A\$79 deliberately matches ServiceM8 Growing, which caps at 150 jobs/mo. Same price, no cap, plus Android. **Do not build a free tier** — ServiceM8's is backed by a decade of scale and would only buy you support load. 14-day trial, card at the end.

Keep the 2% Stripe platform fee (already the default). Leave card surcharge off by default — it's regulated in AU and your Settings warning is correct. Hold annual billing until you have a churn number worth protecting; annual prepay on a product with no error monitoring is a refund liability.

One caution: unlimited AI on Crew at A\$149 is unmodelled, and every tool call ships a full business snapshot. Instrument per-business token cost from customer one and reuse the `seo_monthly_budget_cents` cap pattern you already built.

6. Cut list

You have permission to delete all of this.

Cut outright:

- **The entire SEO Production plugin** — `src/app/(dashboard)/seo`, `src/lib/seo/*`, 7 tables, 13 markdown agents, the `seo_jobs` runner, budget caps, Opportunity Scout, six publish gateways, the GSC OAuth connector, the kireihq GitHub App. Largest single body of code in the repo, serving a market that isn't trades, with zero paying users and almost no shared surface with the job/invoice core. It also carries live operational burden: a Google OAuth app in Testing mode with 7-day token expiry, an `APP_ENCRYPTION_KEY` that can never be rotated, and per-run Anthropic spend. If SEO is a real business, it's a separate product with a separate repo.
- **Content Studio** (`src/app/(dashboard)/content`). Same argument, newer, less sunk cost. A roofer does not want a social content pipeline.
- **Prospects / outbound** (`import_prospects`, `email_prospects`, `bulk_update_prospects`, `convert_prospect`). Cold-outreach tooling for agencies. Also drags you toward spam-compliance obligations you don't want.

- **The agency industry preset and the multi-vertical ambition** in `docs/SEO_AGENCY_PLAN.md`. Keep the plugin machinery (`src/lib/plugins`) — it's cheap and it's your on/off mechanism. Kill the pretence that Kirei serves agencies. Two verticals at 2 active users is how you end up with neither.
- **Expenses, inventory, assets** — check the DB first; if the roofer doesn't use them, ship as stubs or remove. ServiceM8 users complain its inventory module is weak and mostly don't use it; simPRO's depth here is exactly what makes it unusable for small operators. Keep timesheets if it's actually used.

Merge / stop treating as a supported feature:

- **The Quoting Agent as a separate surface** — its own onboarding, settings page, `/api/quoting-agent` endpoint, prompt-caching strategy and memory contract. The main assistant already has `create_quote` and the full registry. Fold the pricing knowledge into the assistant's context and delete the surface. **Keep `quoting_agent_knowledge`** — per-business pricing memory is genuinely valuable.
- **One of the two form builders.** `/onboarding-forms` and `/forms` already share the field engine but ship two builders, two viewers, two MCP tool sets, two buckets, two mental models. Public lead-capture is the one that acquires customers. Merge onboarding into it as a delivery mode.
- **Bookings** — real, but fifth priority behind Xero, billing, offline and Android, and carrying ~10 MCP tools and a settings surface right now.
- **The MCP OAuth authorization server** — painful, because it's the best engineering in the repo. But it exists to serve claude.ai custom connectors, which no trades customer will configure. Header-key auth covers Claude Code and your demos. Keep the code, stop supporting it. (Fixing B10 first is still required if you leave it live.)
- **Smart Organise as eight per-entity proposers and eight buttons.** Keep the undo machinery — it's load-bearing for the assistant. The proposers are polish on a data-quality problem that businesses with 42 lifetime invoices don't have. (It also has a latent bug: the unbounded FK scans at `cleanup.ts:202-218` truncate at PostgreSQL's 1,000-row cap, and `invoices.customer_id` is `on delete set null`, so a blank-fields customer can be hard-deleted leaving an orphaned invoice.)

Docs to delete or fix (30 minutes total, they're actively misleading agents):

- `AGENTS.md` — a sed-substituted fork of `CLAUDE.md`, 41 lines behind, referencing a branch that doesn't exist (`:10`) and "the Codex.ai connector" (`:207-209`) for an OAuth flow built for claude.ai. Replace with a one-line pointer.
- `CLAUDE.md` declares contracts both "shipped" (`:411`) and "Not started — design from scratch" (`:442`). Delete `:442`.
- `CLAUDE.md` `:157-169` documents 7 nav sections (there are 10) and a `NavItem.feature` gating key that **doesn't exist** — the real one is `plugin?: string` (`app-sidebar.tsx:17-24`). Following the documented recipe produces code TypeScript rejects.
- `CLAUDE.md` `:10,12` point at `SCOPE.md` and `docs/invoicer-scope-and-flow.*` — neither exists. `:13` calls the SEO plan the "NEXT BIG INITIATIVE" while the plan itself says "P0-P4 SHIPPED".
- `docs/MOBILE_PARITY_PLAN.md:9-18` says mobile is "worker-only"; `CLAUDE.md:7` says it has full parity. Either refresh it or retile it "historical" and delete the standing rule at `CLAUDE.md:531-534`. A dead rule devalues the live ones.

7. The 2-week plan

One developer. Ordered. Nothing here is optional except where noted.

Week 1 — stop the bleeding, then get paid

Day 1 — Security (must ship together)

- RLS migration for `sms_conversations`, `sms_messages`, `report_sessions` + Realtime filter · **2h** · B1
- `requireCron()` helper across 14 routes; verify `CRON_SECRET` in every Vercel env; redeploy · **1h** · B2
- Worker isolation migration for `sites`, `contacts`, `billing_profiles`, `leads`, `customer_notes` · **2h** · B9
- Delete the `INTERNAL_API_KEY` fallback in `src/lib/api-auth.ts` (check `vercel env ls` first; migrate the Telegram report-sessions integration to a real per-business key) · **1h**
- Set `SENTRY_DSN`; top up `ANTHROPIC_API_KEY` · **15m** · B8

Day 2 — Money correctness

- `addPayment` error handling + amount validation · **30m** · B4
- Mark-paid writes a balancing payments row · **2h** · B5
- Currency: persist `default_currency`, refuse mismatch, change schema default · **3h** · B6
- Gate `booking-db.test.ts` behind `RUN_DB_TESTS=1`; add `.gitattributes` with `* text=auto eol=lf` so the local suite stops being red · **1h** · B7

Day 3 — Error boundaries + confirmations (the two highest-visibility polish items)

- `error.tsx` + `not-found.tsx` at dashboard and root, composed from `PageHeader` + `EmptyState` · **3h** · P2
- `AlertDialog` on the six unconfirmed deletes · **2h** · P1
- `page-header.tsx:30` truncate→break-words; `.ch-table` cursor fix · **15m** · P8/P10

Days 4–5 — Subscription billing · ~2 days · B3 Stripe Billing on the platform account. `plan` + `trial_ends_at` on `businesses`, webhook, paywall in `(dashboard)/layout.tsx`, one billing page. Three prices, no proration. **This is the thing that makes the product sellable.**

Week 2 — make it look and behave like a product you charge for

Day 6 — Email paths

- Route `invoice-reminders`, `quote-followup`, `workorder-complete`, `reminders` through `buildBusinessFrom` + `replyTo` + `getResolvedEmailTemplate` (copy `process-scheduled-sends:119,184`) · **3h** · P4
- Add the portal + pay link to the overdue reminder · **2h** · P3
- Fan out or delete `/api/cron/reminders` · **1h** · P5

Day 7 — Design bugs

- `.ch-pill` and `.ch-stat-delta` dark-mode variants · **2h**
- Delete the three `useIsDark` copies, consume `gradientVars()`, add the dark remap · **2h**
- `themeColor` fix + delete the dead `:root` blue block · **30m**

Day 8 — Design consistency

- Collapse to one status pill primitive; delete `STATUS_GRADIENT` · **3h**
- Desaturate the avatar pool and the dashboard quick actions · **1h**
- Invert the invoice/quote row hierarchy (money loudest, tile gone) · **2h**

Day 9 — Cut day. Delete SEO, Content Studio, Prospects. Fix the six doc contradictions. Write a root README and expand `.env.local.example` from 9 to ~21 variables grouped Required / Payments / Optional — right now no single artifact answers "what must I set to run this?", and that knowledge is unrecoverable once you forget it. · **1 day**

Day 10 — Prove the money path works. Add "Section 27 — Getting paid" to `docs/QA_CHECKLIST.md` (which currently has **zero** occurrences of the word "stripe" across 447 lines) and run it end-to-end in live mode with a real card: Connect onboarding → invoice email pay-link → portal card payment → deposit on quote accept → saved-card autopay → surcharge maths → receipt email. Add `src/lib/__tests__/stripe-amounts.test.ts` and `payment-methods.test.ts` while you're in there (~1h, both are pure functions with zero coverage today). · **1 day**

Immediately after — the two that decide whether you have a business

1. **Upload the Android `.aab` to Play Console.** Hours of work, converts your best competitive claim from theoretical to true.
2. **Build Xero sync.** Weeks, not days. But in Australia the bookkeeper has veto, and until it exists every deal dies in the same place.

Everything on the cut list is what buys you the time to do those two.

Appendix A — All 46 verified findings, in full

Every finding below was raised by a domain specialist and then independently confirmed by a second agent that opened the cited file. Sorted by severity. The main report merges and deduplicates these; this is the unabridged set.

A1. sms_conversations, sms_messages and report_sessions have no RLS at all — and the SMS tables are published to Realtime

Severity: BLOCKER · **Effort:** hours · **Domain:** Security & multi-tenancy

Evidence. supabase/migrations/20260413000002_sms.sql:3-37 (no ENABLE ROW LEVEL SECURITY; lines 36-37 ALTER PUBLICATION supabase_realtime ADD TABLE for both);
supabase/migrations/20260413000003_report_sessions.sql:4-17; src/components/messages/messages-client.tsx unfiltered `.channel("sms-conversations")` subscription on table sms_conversations

Impact. Any authenticated user of any of the 7 tenants can read every business's SMS conversations and message bodies (customer names, E.164 phone numbers, message text) with the anon key, and Realtime — which honours RLS and therefore imposes none here — streams other tenants' conversation rows into any logged-in browser. report_sessions leaks customer ids and property addresses the same way. This is a demonstrable cross-tenant data breach in a product being prepared for sale.

Recommendation. Ship one migration that ENABLEs RLS on all three tables and adds the standard owner-UNION-active-member policy, with `AND NOT public.is_business_worker(business_id)` on the two SMS tables. report_sessions is written only by the admin client (src/app/api/report-sessions/route.ts), so a deny-all policy is sufficient there. Add `filter: business_id=eq.<activeBusinessId>` to the sms-conversations Realtime channel as defence in depth.

Verifier: Verified line by line. supabase/migrations/20260413000002_sms.sql creates sms_conversations (lines 3-14) and sms_messages (15-27) with no ALTER TABLE ... ENABLE ROW LEVEL SECURITY anywhere in the file, and lines 36-37 add both to the supabase_realtime publication. 20260413000003_report_sessions.sql:4-17 likewise creates report_sessions with no RLS. A repo-wide grep for these three table names across all migrations returns only that one file each — no later migration enables RLS or adds a policy. src/components/messages/messages-client.tsx confirms the per-message channel IS filtered by conversation_id, but the sms-conversations channel subscribes to table sms_conversations with event '*' and no filter. Signup is public and /api/auth/signup is whitelisted at src/lib/supabase/middleware.ts:46. One caveat on severity framing: the SMS feature depends on Twilio being configured, so the tables may hold little data today — but the schema defect is unconditional and any future SMS use is cross-tenant readable.

A2. Cron routes fail open when CRON_SECRET is unset — /api/cron/recurring-invoices lets anyone trigger off-session card charges

Severity: BLOCKER · **Effort:** minutes · **Domain:** Functionality & correctness

Evidence. src/app/api/cron/recurring-invoices/route.ts:20-24; src/lib/supabase/middleware.ts:48,68; contrast src/app/api/cron/content-jobs/route.ts:27 and src/app/api/cron/email-leads/route.ts:396-398

Impact. An unauthenticated GET generates invoices for every business with an active schedule and, via generateRecurringInvoice -> chargeInvoiceToSavedCard (src/lib/recurring/generate-invoice.ts:77-80), charges saved cards off-session. The catch-up loop at recurring-invoices/route.ts:46 allows up to 6 cycles per schedule per call. invoice-reminders and quote-followup can likewise be looped to spam every tenant's customers.

Recommendation. Fail closed in all twelve routes: `const s = process.env.CRON_SECRET; if (!s || auth !== \Bearer ${s}) return 401; , additionally accepting req.headers.get("x-vercel-cron")`. Then confirm CRON_SECRET is set in Vercel production and redeploy (env changes do not apply to a running deployment).`

Verifier: Verified verbatim. src/app/api/cron/recurring-invoices/route.ts:22 is `if (process.env.CRON_SECRET && auth !== ...) return 401` — absent env var skips the check entirely. Grepped every cron: the same fail-open shape is at daily-digest:41, invoice-reminders:33, process-scheduled-sends:205, quote-followup:33, recurring-jobs:30, reminders:78, seo-audits:17, seo-gsc-sync:23, seo-jobs:20, seo-reports:16, workorder-complete:29. Only content-jobs/route.ts:27 fails closed (`auth !== Bearer ${CRON_SECRET}` unconditionally) and email-leads/route.ts:396-398 logs+refuses when unconfigured. Confirmed src/lib/supabase/middleware.ts:48 whitelists /api/cron/ and line 68 returns before any auth work, so nothing upstream gates these. booking-reminders/route.ts:19-20 does carry the extra Vercel-cron allowance as claimed. One caveat the finding correctly states as conditional: I could not read the Vercel production env, so whether CRON_SECRET is actually set is unverified — if it is set, the routes are gated today and this is a latent one-env-var-deletion-away failure rather than a live hole.

A3. No error.tsx or not-found.tsx anywhere in the app — 89 pages, 47 notFound() call sites, zero boundaries

Severity: HIGH · **Effort:** hours · **Domain:** UI & UX consistency

Evidence. src/app/(dashboard)/invoices/[id]/page.tsx:42-44; src/app/(dashboard)/leads/[id]/page.tsx:10-12; src/app/(dashboard)/contracts/[id]/page.tsx:26-27; zero matches for error.tsx/global-error.tsx/not-found.tsx under src/app

Impact. A stale bookmark, a deleted record, or an id from another business renders Next's bare black-and-white 404 with no sidebar and no route back into the app. Any throw in a server component renders the bare 'Application error: a server-side exception has occurred' white screen plus a digest hash — and with SENTRY_DSN unset the owner never learns it happened.

Recommendation. Add src/app/(dashboard)/error.tsx (client component with a reset button, rendered inside the shell) and src/app/(dashboard)/not-found.tsx, plus a root src/app/global-error.tsx and src/app/not-found.tsx for the portal and public routes. Compose them from PageHeader + EmptyState so they read as part of the product.

Verifier: Verified. `find src/app -name error.tsx -o -name global-error.tsx -o -name not-found.tsx` returns nothing against 89 page.tsx files, and `grep -ro 'notFound()' src/app | wc -l` = 47 (across 26 files). Cited sites read as claimed (line numbers off by one in two cases): `src/app/(dashboard)/invoices/[id]/page.tsx:42-44` is `} catch { notFound(); }`, `leads/[id]/page.tsx:10-12` same shape, `contracts/[id]/page.tsx:26-27` is `let contract; try { contract = await getContract(id); } catch { notFound(); }` then `if (!contract) notFound();`. So every 404 and every unhandled server-component throw falls through to Next's unstyled built-in pages. Severity corrected from blocker to high: it degrades every error path and there is no Sentry to catch it, but it does not block charging money and no evidence was gathered that it is currently firing on a hot path.

A4. Destructive one-click deletes with no confirmation on expenses, assets, SEO reports/connections and Content brands

Severity: HIGH · **Effort:** hours · **Domain:** UI & UX consistency

Evidence. `src/components/expenses/expenses-view.tsx:86-91,136`; `src/components/assets/assets-view.tsx:57,107`; `src/components/seo/seo-reports.tsx:40,87`; `src/components/seo/seo-connections.tsx:92,297`; `src/components/content/content-brands-view.tsx:74,199`; `src/lib/actions/assets.ts:72-77`; contrast `src/components/leads/leads-client.tsx:445-459`

Impact. A misclick on a 16px trash icon in a dense table row — sitting immediately beside the receipt-view icon in expenses — permanently destroys a financial record with no confirm and no undo. Expenses feed job costing, so a silent loss corrupts margin figures for the one business actually relying on this.

Recommendation. Add the AlertDialog pattern `leads-client.tsx` already uses: a `deleteId` state plus one shared dialog per view. Six files, the same mechanical edit.

Verifier: Verified at all six cited sites, and I checked each handler body for a `window.confirm` — there is none. `expenses-view.tsx:136` calls `handleDelete(e.id)` whose body (lines 86-91) goes straight into `startTransition → deleteExpense`. Same for `assets-view.tsx:107 → handleDelete` (line 57), `seo-reports.tsx:87 → handleDelete` (line 40), `seo-connections.tsx:297 → handleDelete` (line 92), `content-brands-view.tsx:199 → remove` (line 74, direct `deleteContentBrand`), `content-campaigns-view.tsx:149 → remove`. `grep -c AlertDialog` returns 0 in all of them. The delete is a hard row delete: `src/lib/actions/assets.ts:72-77` is `.delete().eq("id", id).eq("business_id", businessId)`. `leads-client.tsx:445-459` does gate deletes behind an AlertDialog reading "This can't be undone.", so the inconsistency is real. Severity held at high — this is the one finding in the set that destroys user data.

A5. Cron routes email a tenant's customers from the platform's Crown Roofers address, bypassing `buildBusinessFrom` and `email_templates`

Severity: HIGH · **Effort:** hours · **Domain:** Performance & reliability

Evidence. `src/app/api/cron/reminders/route.ts:17,138,175-178`; `invoice-reminders/route.ts:19,130`; `quote-followup/route.ts:19,132`; `workorder-complete/route.ts:19,128`; `daily-digest/route.ts:18,183-184`. Contrast `process-scheduled-sends/route.ts:119,184` and `src/lib/recurring/generate-invoice.ts:119-125`.

Impact. A second business enabling the invoice-reminders or quote-followup agent sends its customers overdue-payment demands and quote chases from `Crown Roofers <invoices@crownroofers.com.au>` (the production RESEND_FROM_EMAIL per CLAUDE.md), with no reply-to pointing back at the actual sender. The per-business email_templates built in June are silently ignored on these paths.

Recommendation. Replace the module-level FROM in the four customer-facing routes with `buildBusinessFrom({ name: biz.name, localPart: "invoices"|"quotes"|"jobs" })` plus `replyTo: biz.email`, and route bodies through `getResolvedEmailTemplate(sb, business_id, type)` as process-scheduled-sends does. Remove the hardcoded `info@crownroofers.com.au` replyTo at reminders/route.ts:176. Add a test asserting no cron route references RESEND_FROM_EMAIL directly.

Verifier: Verified all five files. `src/app/api/cron/reminders/route.ts:17` is `const FROM = process.env.RESEND_FROM_EMAIL ?? "Crown Roofers <noreply@crownroofers.com.au>"`; invoice-reminders:19, quote-followup:19, workorder-complete:19, daily-digest:18 are all `?? "Kirei <noreply@resend.dev>"`. All pass FROM straight into `resend.emails.send({ from: FROM, ... })` (invoice-reminders:130, quote-followup:132, workorder-complete:128, daily-digest:183) with no replyTo and no getResolvedEmailTemplate call. process-scheduled-sends:119,184 does correctly use `buildBusinessFrom({ name: business.name, localPart: "invoices"|"quotes" })`, as does `src/lib/recurring/generate-invoice.ts:123-124` (which also sets replyTo: biz.email). Two corrections to the original finding: (a) daily-digest sends to `biz.email` (route.ts:184) — the business owner, not their customers — so it is internal-only and not customer-visible; the customer-facing paths are invoice-reminders, quote-followup, workorder-complete and reminders. (b) reminders/route.ts:176 does set `replyTo: "info@crownroofers.com.au"` on the customer email — hardcoded to Crown Roofers, which makes it worse, not better. Severity lowered from blocker to high: it does not block taking money, and it is dormant until a second tenant enables one of these optional agents, but it is genuinely customer-visible when it fires.

A6. SEO job runner claims jobs without a lock — overlapping ticks double-bill and lose the cost counter

Severity: HIGH · **Effort:** hours · **Domain:** Performance & reliability

Evidence. `src/app/api/cron/seo-jobs/route.ts:14,26-32,40-41`; `src/app/api/cron/content-jobs/route.ts:6-11,36-39`; `src/lib/seo/engine.ts:344-351` (cost_cents read-modify-write), 101-114 (seoSpendStatus).

Impact. A step longer than 120s is re-picked by the next tick while still running: both invocations pay for the same Claude call and both write step+1 from the same stale read, overwriting a pipeline stage. The stale-read cost increment means only one charge is recorded, so `seo_monthly_budget_cents` leaks. Latent today only because `ANTHROPIC_API_KEY` returns 401; it becomes live uncapped spend the moment the key is topped up.

Recommendation. Add a `claim_seo_jobs(max_jobs, lease_secs)` SQL function mirroring `claim_content_jobs`, add `locked_until` to `seo_jobs`, and switch the runner to the RPC. Make the `cost_cents` write an atomic SQL increment rather than a stale-read add.

Verifier: Confirmed in full, and the codebase itself already documents it. seo-jobs/route.ts:26-32 plain-SELECTs seo_jobs where status in ('queued','running') and then :40 does `update({status:"running"})` before calling advanceContentJob — no lease, no SKIP LOCKED. content-jobs/route.ts:36 uses `rpc("claim_content_jobs", { max_jobs, lease_secs })` and its header comment at lines 6-11 names the SEO cron explicitly as the one that 'two overlapping ticks pick up the same job and run (and bill) the same step twice'. Both are `*/2 * * * *` in vercel.json with maxDuration = 300 (seo-jobs/route.ts:14). The cost write in src/lib/seo/engine.ts is `cost_cents: (job.cost_cents ?? 0) + costCents` from the pre-step read, and seoSpendStatus sums those rows, so the budget cap does under-count under concurrency. Line number correction: the cost write is at engine.ts:349, not 346; seoSpendStatus is at engine.ts:101-114.

A7. Worker isolation never extended to sites, contacts, billing_profiles, leads or customer_notes

Severity: HIGH · **Effort:** hours · **Domain:** Security & multi-tenancy

Evidence. supabase/migrations/20260430000001_worker_role_and_isolation.sql (is_business_worker appears only for customers/products/invoices/quotes/payments/reports); 20260418000001_account_site_portfolio.sql:51 sites_all, :148 billing_profiles_all; 20260428000002_unified_contacts.sql:71 contacts_all (the live contacts policy); 20260412000001_leads.sql:42 leads_business_access; 20260403000001_customer_hub.sql:51 customer_notes

Impact. A worker — the role CLAUDE.md documents as hard-isolated — is blocked from `customers` but can SELECT/INSERT/UPDATE/DELETE every row of sites (all serviced property addresses), contacts (all contact names/emails/phones), billing_profiles, leads (the whole sales pipeline) and customer_notes for the business, via the mobile app's Supabase client or a raw PostgREST call. Half the isolation guarantee the schema exists to provide is not real, and the gap is invisible from the app UI.

Recommendation. One migration replacing those five policies with the same predicate plus `AND NOT public.is_business_worker(business_id)`. Then add a CI check that enumerates tables with a business_id column and asserts each has at least one policy referencing is_business_worker, so the next table added before/after a role migration can't silently miss it.

Verifier: Confirmed with one citation correction and one sub-claim refuted. The *_no_workers policies in 20260430000001_worker_role_and_isolation.sql cover exactly customers (:76), products (:97), invoices (:118), quotes (:139), payments (:160) and reports (:181) — grep for is_business_worker in that file matches nothing else. sites_all (20260418000001:51), billing_profiles_all (:148) and customer_notes (20260403000001:51) are plain owner-UNION-active-member FOR ALL policies. The contacts citation needs updating: 20260418000001:86 contacts_all was superseded by 20260428000002_unified_contacts.sql:71, which recreates contacts_all with the same worker-blind predicate — so the defect holds, at a different line. leads_business_access (20260412000001:42) also has no worker clause. REFUTED sub-claim: the missing WITH CHECK on the leads policy is not a defect — Postgres uses the USING expression as the check when WITH CHECK is omitted. Only 12 of 100 migrations reference is_business_worker at all, and none of them touch these five tables.

A8. OAuth consent flow mints a permanent admin-scope API key for any active member, including workers

Severity: HIGH · **Effort:** hours · **Domain:** Security & multi-tenancy

Evidence. `src/app/api/oauth/authorize/route.ts` `listBusinesses` (`business_members` query with no role filter) and the POST membership re-check; `src/app/api/oauth/token/route.ts` insert into `business_api_keys` with `scopes: ["admin"]` and no `expires_at`; `src/lib/actions/api-keys.ts` `if (!canManageSettings(role)) throw new Error("Forbidden")`; `src/lib/assistant/scopes.ts` case "worker": return null

Impact. A worker or viewer who completes the claude.ai connector flow receives a never-expiring `inv_*` key with the admin wildcard, giving read/write access to every customer, invoice, quote and payment in the business through the ~199 MCP tools — the precise privilege escalation the assistant scope layer was written to prevent. It is an insider-only path (the actor must already be an active member), which is why this is high rather than blocker.

Recommendation. Join role in `listBusinesses` and drop worker (and ideally viewer) memberships; replace the hardcoded `["admin"]` with `assistantScopesForRole(role)`; set `expires_at` (e.g. 90 days) on the minted key and return `expires_in` so clients refresh.

Verifier: Confirmed exactly as described. `src/app/api/oauth/authorize/route.ts` `listBusinesses` selects `business_members` rows filtered only on `user_id` and `status='active'` — `role` is never read; the POST handler re-checks only membership in that same unfiltered list before inserting an `oauth_codes` row with scope 'admin'. `src/app/api/oauth/token/route.ts` then inserts a `business_api_keys` row with `scopes: ["admin"]`, no `expires_at`, and returns it as `access_token` with no `expires_in`. The contrast the finding draws is real: `src/lib/actions/api-keys.ts` gates `createApiKey` on `canManageSettings(role)` and throws `Forbidden` otherwise, and `src/lib/assistant/scopes.ts` `assistantScopesForRole` returns null for 'worker' with a comment explicitly noting the admin client bypasses RLS. `src/lib/mcp/context.ts` confirms MCP tools run on the service-role client scoped only by `business_id`.

A9. Cron routes are unauthenticated when CRON_SECRET is unset — including the one that charges saved cards

Severity: HIGH · **Effort:** minutes · **Domain:** Security & multi-tenancy

Evidence. `src/app/api/cron/recurring-invoices/route.ts:22` and the same `process.env.CRON_SECRET &&` short-circuit at `daily-digest:41`, `invoice-reminders:33`, `process-scheduled-sends:205`, `quote-followup:33`, `recurring-jobs:30`, `reminders:78`, `seo-audits:17`, `seo-gsc-sync:23`, `seo-jobs:20`, `seo-reports:16`, `workorder-complete:29`; contrast `src/app/api/cron/content-jobs/route.ts:27` and `email-leads/route.ts:396-398`

Impact. If `CRON_SECRET` is ever unset, renamed or misspelled on Vercel, anyone who guesses the URL can drive `/api/cron/recurring-invoices` — generating invoices and off-session charging customers' saved Stripe cards on a LIVE Stripe integration — or hammer `invoice-reminders/quote-followup` to send mail from the business's verified domain. The endpoints return 200 either way, so the misconfiguration is silent.

Recommendation. Factor a single `requireCron(req)` helper that reads the secret once and returns 401 when it is missing OR mismatched, then use it in every cron route including `booking-reminders`. Add a lint or CI grep forbidding the `process.env.CRON_SECRET &&` form.

Verifier: Confirmed. A grep of `src/app/api/cron` for `CRON_SECRET` shows the fail-open form `if (process.env.CRON_SECRET && auth !== `Bearer ${...}`) in daily-digest:41, invoice-reminders:33, process-scheduled-sends:205, quote-followup:33, recurring-invoices:22, recurring-jobs:30, reminders:78, seo-audits:17, seo-gsc-sync:23, seo-jobs:20, seo-reports:16, workorder-complete:29 – twelve routes. Only content-jobs:27 fails closed ( if (auth !== `Bearer ${process.env.CRON_SECRET}`)) and email-leads:396-398 which logs and refuses when unset. booking-reminders:19-20 is a third variant worth including in the fix. /api/cron/ is unconditionally public in src/lib/supabase/middleware.ts. Severity caveat the original agent did not state: the exposure is contingent on CRON_SECRET actually being absent or misnamed on Vercel, which I could not verify from the repo — but the fail-open pattern itself is unconditional and the fix costs minutes.`

A10. Payment-chasing and follow-up emails are sent from one global From address, bypassing the per-business sender and template system

Severity: HIGH · **Effort:** hours · **Domain:** Functionality & correctness

Evidence. `src/app/api/cron/invoice-reminders/route.ts:19,99-126,129-134`; `quote-followup/route.ts:19,133`; `workorder-complete/route.ts:19,128`; `daily-digest/route.ts:18,183`; contrast `src/lib/recurring/generate-invoice.ts:108-126`

Impact. Business B's customer receives an overdue-payment demand for Business B's invoice from another business's verified domain, with replies going there. The per-business email-template feature does not apply to any of these sends. The highest-intent email in the product — the overdue chase — offers the customer no way to pay.

Recommendation. Replace `resend.emails.send({ from: FROM })` with `sendEmail({ from: buildBusinessFrom({ name: biz.name, localPart: "invoices" }), replyTo: biz.email })` in all four routes and route bodies through `getResolvedEmailTemplate`. For invoice-reminders, reuse the `getOrMintToken` helper pattern from `src/lib/recurring/generate-invoice.ts:131-144` and include the portal link plus `/api/stripe/checkout?invoice=...&token=...`.

Verifier: Verified across all four routes: the module-level `const FROM = process.env.RESEND_FROM_EMAIL ?? "Kirei <noreply@resend.dev>"` exists at `invoice-reminders:19`, `quote-followup:19`, `workorder-complete:19`, `daily-digest:18`, and is used as `from: FROM` at `invoice-reminders:130`, `quote-followup:133`, `workorder-complete:128`, `daily-digest:183` — plus `reminders/route.ts:138,175`. None sets `replyTo`. CLAUDE.md documents production `RESEND_FROM_EMAIL` as the Crown Roofers address. Also verified the secondary claim: the invoice-reminders body (lines 99-126) is inline HTML built in-route, never touches `getResolvedEmailTemplate`, and contains zero links — no portal link and no pay link, only prose asking the customer to arrange payment. The `btn` helper is imported at line 14 and never used. Contrast confirmed: `src/lib/recurring/generate-invoice.ts:119-125` does it correctly with `buildBusinessFrom + replyTo + template + payUrl`.

A11. addPayment discards the payments-insert error, so a rejected payment silently records as a no-op

Severity: HIGH · **Effort:** minutes · **Domain:** Functionality & correctness

Evidence. src/lib/actions/invoices.ts:339-372 (insert at :351, unchecked updates at :372 and :417)

Impact. On any insert failure (RLS denial, CHECK violation, bad date string) the action completes without error, the UI reports success and revalidates, and the user believes a payment was recorded when nothing was written. Money-recording failure presented as success.

Recommendation. `const { error: payErr } = await tbl(...).insert(...); if (payErr) throw payErr;` and the same on the updates at 372 and 417. Add `if (!Number.isFinite(payment.amount) || payment.amount <= 0) throw new Error(...)` at the top.

Verifier: Verified exactly. src/lib/actions/invoices.ts:351 is a bare `await tbl(supabase, "payments").insert({...})` — no destructure, no throw. Execution falls straight through to the recompute at 357-372 which re-reads the payments table, so a failed insert yields the pre-existing amount_paid written back over itself. The two subsequent updates at 372 and 417 are likewise unchecked. There is no validation of payment.amount anywhere in the function (it reads `total, amount_paid` at 345-348 and never inspects the incoming amount). This is the same shape as the `.or()` incident in CLAUDE.md's Known traps. Note the trigger requires the insert to actually fail, which for an owner-role user is uncommon — but the function is also reachable by editor-role users and the MCP/assistant surface.

A12. Recording a payment on an invoice marked paid via the status dropdown silently reverses it to partial

Severity: HIGH · **Effort:** hours · **Domain:** Functionality & correctness

Evidence. src/lib/actions/invoices.ts:96-99 vs :357-372; guard asymmetry at :409-415

Impact. Mark a \$10,000 invoice paid from the status dropdown (amount_paid=10000, no payments row), then record any later payment against it: amount_paid is rewritten to just that payment's value and status drops to partial. A settled invoice reads as outstanding on the dashboard and becomes a target for the overdue-reminder cron, which will email the customer chasing money already paid.

Recommendation. Make payments the single source of truth: have updateInvoice's mark-paid branch insert a balancing payments row (method "Manual — marked paid") instead of writing amount_paid directly, so every recompute agrees. At minimum, never let addPayment lower amount_paid without an explicit reversal, and extend the draft/cancelled guard from :412 to the invoice's own status write at :370.

Verifier: Verified — the two writers genuinely disagree. src/lib/actions/invoices.ts:96-99 (updateInvoice) sets `patch.amount_paid = cur.total` when status flips to paid and writes no payments row. addPayment at :357-372 then recomputes amount_paid purely as directSum + childSum read from the payments table and overwrites the column, with newStatus at :370 collapsing to paid/partial. Confirmed the asymmetry the finding flags: the parent-rollup branch at :409-415 explicitly refuses to fight `cancelled / draft`, while the invoice's own status update at :370-372 has no such guard. Note the recompute IS deliberate — CLAUDE.md's Known traps document it as the fix for progress-invoice rollup, so the fix must preserve that behaviour rather than revert it.

A13. Every business defaults to GBP, so the live Stripe checkout charges the wrong currency until someone changes a setting

Severity: HIGH · **Effort:** hours · **Domain:** Functionality & correctness

Evidence. supabase/migrations/001_initial_schema.sql:20; src/app/api/stripe/checkout/route.ts:68,70,83; src/lib/stripe-charge.ts:59; src/app/api/stripe/webhook/route.ts:54-64; src/components/settings/settings-client.tsx:353-360

Impact. A business that never opens the currency setting has its customers charged the invoice's numeric total in pounds on a non-GBP connected account, with an FX conversion on settlement. Nothing in the codebase detects the mismatch. This is a live risk on the first real use of a payment path that has never been exercised.

Recommendation. Persist account.default_currency in handleAccountUpdated and treat it as the source of truth for charge currency (falling back to businesses.currency). Add a hard refusal in checkout/route.ts and stripe-charge.ts when the resolved currency differs from the connected account's default. Change the schema default off GBP and force the currency choice at signup.

Verifier: Every cited line verified: supabase/migrations/001_initial_schema.sql:20 `currency text not null default 'GBP'`; src/app/api/stripe/checkout/route.ts:68 `(business.currency || "GBP").toLowerCase()` feeding line items at :83 and the fee at :70; src/lib/stripe-charge.ts:59 identical for off-session autopay; invoice-reminders/route.ts:24 formats with en-GB. handleAccountUpdated (webhook/route.ts:54-64) does receive the Stripe Account and persists only country — it never reads default_currency, and no code anywhere compares the two. I also checked the signup path: it never sets currency, so every business inherits GBP; the only way to change it is the Settings → Business select (src/components/settings/settings-client.tsx:353-360, defaulting to GBP). One correction to the impact framing: currency is applied app-wide, not only at checkout, so a business on the wrong setting would see £ throughout its invoices and PDFs and would likely notice before a customer paid. The durable defect is the absent guard against a currency the connected account does not settle in, plus a default that is wrong for the product's actual market.

A14. npm test writes to the production database, into whichever business happens to be first

Severity: HIGH · **Effort:** hours · **Domain:** Test coverage & quality

Evidence. src/lib/booking/**tests**/booking-db.test.ts:17,29-33,36-40,42-63; src/lib/booking/**tests**/setup.ts:2-4; vitest.config.ts test.setupFiles

Impact. Running the documented `npm test` command mutates a real tenant's booking data. An interrupted run leaves a phantom VITEST_TMP resource and year-2099 appointments visible in a paying customer's UI. CI never exercises this path because CI has no Supabase secrets, so these tests only ever run against production.

Recommendation. Gate the DB-backed suite behind an explicit opt-in exactly as the live-API suite already does (`process.env.RUN_LIVE_API === "1"` at src/lib/mcp/**tests**/live-api.test.ts:23) — e.g. `RUN_DB_TESTS=1`. Better: create a dedicated throwaway business in beforeAll rather than `limit(1)`, and wrap cleanup in try/finally.

Verifier: Verified in full. `src/lib/booking/tests/booking-db.test.ts:29` does `admin.from("businesses").select("id").limit(1).single()` and `:31-33` inserts a `booking_resources` row named `VITEST_TMP` into that arbitrary business; `:42-63` inserts 8 concurrent appointments plus a back-to-back one. The gate is `ready = Boolean(URL && SERVICE && ANON)` (`:17`) and `vitest.config.ts` sets `setupFiles: [".src/lib/booking/__tests__/setup.ts"]`, which does `config({ path: ".env.local" })`. I confirmed `.env.local` contains all three keys, and `vitest` reported `injected env (33) from .env.local` on the run I did do. `CLAUDE.md` confirms one Supabase project, so those keys are production. Cleanup is `afterAll` only (`:36-40`), so an interrupt orphans the rows. I deliberately did NOT run the DB suite to avoid the very mutation described.

A15. Seven plugin pages import zero Kirei primitives and hand-roll a duplicate Stat tile, empty state and table idiom

Severity: MEDIUM · **Effort:** hours · **Domain:** UI & UX consistency

Evidence. `src/components/assets/assets-view.tsx:166`; `src/components/inventory/inventory-view.tsx:153`; `src/components/expenses/expenses-view.tsx:196`; `src/components/seo/seo-site-hub.tsx:250`; dashed empty states at `inventory-view.tsx:76`, `assets-view.tsx:84`, `expenses-view.tsx:115`

Impact. Clicking Invoices then Expenses shows two visually different products — gradient KPI tiles and pill-tabbed cards versus flat boxes and a dense table. The hand-rolled versions also miss what the shared primitives carry for free, notably dark-mode tokens.

Recommendation. Replace the four local Stat functions with `StatTile` and the four dashed blocks with `EmptyState`. This deletes code rather than adding it; it can be done incrementally, one view per PR.

Verifier: Facts verified. Four separate local `function Stat(...)` definitions exist at `assets-view.tsx:166`, `inventory-view.tsx:153`, `expenses-view.tsx:196`, `seo-site-hub.tsx:250`. The identical dashed empty state `rounded-xl border border-dashed border-border bg-card/50 p-12 text-center` appears at `inventory-view.tsx:76`, `assets-view.tsx:84`, `expenses-view.tsx:115`, `seo-site-hub.tsx:262`. All three views render `.ch-table` (`inventory:84`, `assets:92`, `expenses:123`) while 23 files import from `@/components/ui/kirei` and none of the plugin views do. Severity corrected from high to medium: this is a genuine, measurable inconsistency but it is visual polish, not a defect — no data loss, no broken flow. The plugin pages are also default-off, so most prospects never see them.

A16. Status-pill colour maps are light-mode only, while the dark-mode toggle ships on every page of the header

Severity: MEDIUM · **Effort:** hours · **Domain:** UI & UX consistency

Evidence. `src/components/layout/app-header.tsx:61-69`; `src/components/work-orders/job-portfolio-client.tsx:101-109`; `src/components/ui/kirei/pill.tsx:44-51`; `src/components/assets/assets-view.tsx:21`; `src/components/prospects/prospects-view.tsx:20-23`; `src/app/audit/report/[id]/page.tsx:8-11`

Impact. In dark mode these pills keep their light pastel backgrounds and mid-tone text against a near-black canvas. The work-order detail screen is the most-used surface for a tradie, and `/audit/report/[id]` is customer-facing lead-gen output.

Recommendation. Replace each local STATUS_COLORS/STATUS_TONE/SEV_TONE map with , which already covers draft/assigned/in_progress/submitted/reviewed/completed/cancelled. Add the two or three missing keys (in_repair, the severity tones) to pill.tsx rather than re-deriving them per file.

Verifier: Verified. src/components/layout/app-header.tsx:61-69 renders an always-visible ghost icon Button calling setTheme(theme === "dark" ? "light" : "dark"). job-portfolio-client.tsx:101-109 defines STATUS_COLORS with bg-blue-100/text-blue-700 etc. and no dark: variants, while src/components/ui/kirei/pill.tsx:44-51 already carries dark:bg-blue-900/40 dark:text-blue-200 for the same tones. Light-only maps also confirmed at assets-view.tsx:21, prospects-view.tsx:20-23, and src/app/audit/report/[id]/page.tsx:8-11. I did not visually render dark mode, so the specific description of how bad it looks is inference, but the missing dark: variants are factual. Severity corrected from high to medium — cosmetic, no data or flow impact.

A17. PageHeader clips the page title with `truncate` — on every page, against the documented break-words rule

Severity: MEDIUM · **Effort:** minutes · **Domain:** UI & UX consistency

Evidence. src/components/layout/page-header.tsx:30 (h1 with `truncate`), :29 (min-w-0 wrapper), :23 (flex-wrap parent); CLAUDE.md web-reskin note on break-words vs truncate

Impact. On narrow viewports the actions slot competes for width and the min-w-0 title container shrinks, so titles ellipsis with no tooltip. Worst on detail pages that pass a record name as the title, where a customer or contract name becomes unreadable.

Recommendation. Change `truncate` to `break-words` on line 30 and verify against a long customer name at 375px.

Verifier: Verified with a two-line offset in the citation: the h1 is at src/components/layout/page-header.tsx:30, not :32 — `<h1 className="text-3xl font-bold tracking-tight truncate">{title}</h1>` — inside a min-w-0 div at :29, whose parent flex-wrap container is at :23. CLAUDE.md's design-system section does state list pages use break-words "(not truncate — user wants text to wrap, not clip, on mobile)", so this is a documented-preference violation in the single most-rendered dashboard component. I did not render at 375px, so the specific clipping examples are inference; the truncate class is not.

A18. Plugin list pages ship without search, status tabs or Smart Organise, breaking the documented standing rule

Severity: MEDIUM · **Effort:** days · **Domain:** UI & UX consistency

Evidence. grep count 0 for a search placeholder in src/components/{expenses,assets,inventory,timesheets}/*-view.tsx and src/components/contracts/contracts-client.tsx; CleanupButton imported in only 8 files; src/components/leads/leads-client.tsx:308-317

Impact. Once a business accumulates a few hundred expense rows or stock items the page becomes an unfilterable wall with no way to find a record. A prospect comparing against ServiceM8 will try to search on the first list they open.

Recommendation. Port the leads-client search pattern (a `search` state plus a useMemo filter and the same styled Input) into the five views, and add to their PageHeader actions where a proposer exists.

Verifier: Verified. `grep -c 'placeholder="Search'` returns 0 in expenses-view.tsx, assets-view.tsx, inventory-view.tsx, timesheets-view.tsx and contracts-client.tsx. CleanupButton is imported in exactly 8 consumers — contacts, customers, invoices, leads, products, quotes, team, work-orders — against CLAUDE.md's claim that it "lives in every list page's ", so the documented rule has drifted. leads-client.tsx:308-317 is the search pattern to copy. I did not confirm the absence of pagination on those tables beyond seeing the plain `.map` over the full array, so treat the 'no pagination' half as weaker than the search half.

A19. /api/cron/reminders is hardcoded to one business UUID and one Telegram chat

Severity: MEDIUM · **Effort:** hours · **Domain:** Performance & reliability

Evidence. src/app/api/cron/reminders/route.ts:14,16,68; vercel.json crons entry {"path": "/api/cron/reminders", "schedule": "0 20 * * *"}.

Impact. A daily production cron that is structurally single-tenant. Every business that signs up gets zero schedule reminders and zero worker job emails from it, with no error surfaced. The Telegram summary also goes to one hardcoded chat id.

Recommendation. Either fan out over `business_agent_installs` the way invoice-reminders/route.ts:43 does (moving the Telegram chat id to a per-business settings column), or delete the route and its vercel.json entry and keep the capability in the per-business agents store.

Verifier: Exact match. reminders/route.ts:14 `const BUSINESS_ID = process.env.AGENT_BUSINESS_ID ?? "ff3a47f3-54b0-45e3-b7a9-69ddc9fa787e"`, :16 `const CHAT_ID = process.env.TELEGRAM_CHAT_ID!`, and the only query scopes on `.eq("business_id", BUSINESS_ID)` at :68. vercel.json schedules it daily at `0 20 * * *`. There is no business_agent_installs fan-out anywhere in the file, unlike invoice-reminders/route.ts:43-47. Severity kept at high in structural terms but note it is silent rather than damaging — nothing breaks for other tenants, they simply get nothing.

A20. Recurring-invoice cron can double-charge a saved card: next_run_on advances only after the loop, no Stripe idempotency key

Severity: MEDIUM · **Effort:** hours · **Domain:** Performance & reliability

Evidence. src/app/api/cron/recurring-invoices/route.ts:30-34 (unbounded select, no maxDuration in file), :46-69 (loop), :71-76 (next_run_on written after loop); src/lib/recurring/generate-invoice.ts:77-80; src/lib/stripe-charge.ts:73-80 (paymentIntents.create with confirm:true, no idempotencyKey).

Impact. If the invocation is killed after a card is charged but before next_run_on advances, the next daily run regenerates the invoice and charges the same customer again, and Stripe will not dedupe it. Risk grows as the unbounded schedule list outgrows one invocation.

Recommendation. Advance `next_run_on` inside each iteration immediately after a successful generate rather than after the loop; pass a deterministic Stripe idempotency key (`${recurring_invoice_id}:${runOn}`) through `chargeInvoiceToSavedCard`; add `export const maxDuration = 300` and a bounded `.limit()` with a cursor.

Verifier: The cited mechanics are accurate. `recurring-invoices/route.ts:46-69` runs the catch-up while loop calling `generateRecurringInvoice` (which charges the saved card at `src/lib/recurring/generate-invoice.ts:77-80`), and `next_run_on` is only written at `:71-76` after the loop completes. There is no `export const maxDuration` in the file (`invoice-reminders/route.ts:17` sets 60, `content-jobs/route.ts:18` sets 300). The schedule query at `:30-34` is `select("")` with no `.limit()`. I confirmed `src/lib/stripe-charge.ts:73-80` calls `stripe.paymentIntents.create(..., { confirm: true })` with no idempotencyKey, so Stripe will not dedupe a repeat. Two mitigations the finding omits, which is why I lower severity to medium: the in-loop try/catch at `:63-66` deliberately leaves `next_run_on` unadvanced on generate failure with the comment 'so it retries next run', so the ordinary error path is a considered choice, and the window is narrow — the charge is the last thing that can throw before the write (the email path at `generate-invoice.ts:83-89` is itself caught). The defect requires the function to be killed mid-flight (timeout, deploy) between the successful charge and the `next_run_on` update.

A21. Smart Organise customer cleanup can propose deleting a customer that has invoices once a business exceeds PostgREST's row cap

Severity: MEDIUM · **Effort:** hours · **Domain:** Performance & reliability

Evidence. `src/lib/actions/cleanup.ts:79-87` (FK columns), `137-141` (unbounded customers select), `202-218` (unbounded sequential ref scans), `224-241` (delete-vs-archive guard + `isEmpty`); `supabase/migrations/001_initial_schema.sql:83,108` (`on delete set null`).

Impact. With more than ~1,000 rows in any referencing table the reference set is silently truncated, so a blank-fields customer that an old invoice points at can be proposed for hard delete on the `/customers` page; the FK is SET NULL so the invoice is orphaned rather than the delete being blocked. The six scans are also sequential rather than `Promise.all`, stacking six round trips into the button's latency.

Recommendation. Scope the reference lookups to the delete candidates with `.in(fc.column, candidateIds)` (bounded by construction), or paginate with `.range()` until a short page returns so truncation can never read as 'no references'. Run them via `Promise.all` either way.

Verifier: The code is as described: `cleanup.ts:202-218` loops the six `CUSTOMER_FK_COLUMNS` (defined `:79-87`) doing an unbounded, unpaginated, sequential `.select(fc.column).eq("business_id", ...).not(fc.column, "is", null)` into `idsRefByFk`, and that set is the sole delete-vs-archive guard at `:224-229`. The customers pull at `:137-141` is also `select("*")` unbounded. I checked the FK direction too: `invoices.customer_id` is `on delete set null` (`supabase/migrations/001_initial_schema.sql:83,108`; also `010_work_orders.sql:16`, `20260412000001_leads.sql:32`), so the database will NOT refuse the delete — the invoice survives orphaned. One correction that materially narrows the blast radius: a delete is only proposed for rows that also pass the `isEmpty` test at `:233-241`, which requires email, phone, address, city, postcode, country, company and notes all to be blank. So the affected row must be BOTH invoiced beyond the 1,000-row truncation AND have no contact details at all. Real, but far rarer than 'a customer referenced only by invoice #1,001'. Severity lowered accordingly.

A22. The retired /api/agent is still live and is still what the in-app floating assistant calls

Severity: MEDIUM · **Effort:** days · **Domain:** Performance & reliability

Evidence. `src/app/api/agent/route.ts` (2,151 lines, exists); `src/components/agent/agent-panel.tsx:40,246`; `src/components/layout/dashboard-shell.tsx:13-14,82`; `src/app/api/assistant/route.ts` (exists).

Impact. Two divergent agent implementations ship. New MCP tools reach `/api/assistant` automatically but never the floating panel, so the two assistants answer the same question with different capabilities — the drift CLAUDE.md's standing rule exists to prevent.

Recommendation. Point `agent-panel.tsx` at `/api/assistant` (matching the block-shaped `MessageParam` contract the assistant and mobile agents share), keep the panel UI, then delete `src/app/api/agent/route.ts`. Confirm first whether the assistant PR stack already does this before duplicating the work.

Verifier: Verified in this working tree. `src/app/api/agent/route.ts` exists and is 2,151 lines; `src/components/agent/agent-panel.tsx:246` does `fetch("/api/agent", ...)` (with a comment at :40 'API message format (sent to /api/agent)'); `dashboard-shell.tsx:13-14` dynamic-imports `AgentPanel` and mounts it at :82 on every dashboard page. `src/app/api/assistant/route.ts` also exists. So both engines ship. One caveat worth stating in the report: the assistant rebuild PR stack (#398-402) is recorded as unmerged, so this may be branch state rather than a regression — but on this branch, which is the polish branch, the drift is real and CLAUDE.md already describes `/api/agent` as replaced.

A23. Cron routes fail OPEN if CRON_SECRET is unset — the auth check is conditional on the variable existing

Severity: MEDIUM · **Effort:** minutes · **Domain:** Performance & reliability

Evidence. `src/app/api/cron/{reminders:78, daily-digest:41, invoice-reminders:33, quote-followup:33, workorder-complete:29, recurring-invoices:22, recurring-jobs:30, process-scheduled-sends:205, seo-jobs:20, seo-audits:17, seo-reports:16, seo-gsc-sync:23}/route.ts`; contrast `content-jobs/route.ts:27` and `email-leads/route.ts:396-398`.

Impact. In any environment missing `CRON_SECRET` these become publicly callable GETs with no body. `/api/cron/invoice-reminders` sends real payment-demand emails to real customers, `/api/cron/recurring-invoices` charges real saved cards, `/api/cron/seo-jobs` spends Anthropic credits. Since local dev and previews point at the production Supabase project, a preview environment without the var is enough to reach production data.

Recommendation. Extract one `assertCronAuth(req)` helper returning 500 when `CRON_SECRET` is unset and 401 on mismatch — the `email-leads` shape — and use it in all fourteen routes. Verify `CRON_SECRET` is present in every Vercel environment, not just production.

Verifier: Confirmed by grepping every cron route. The conditional guard `if (process.env.CRON_SECRET && auth !== ...)` appears in twelve routes, not eleven — the finding's list omits reminders/route.ts:78. Full list: reminders:78, daily-digest:41, invoice-reminders:33, quote-followup:33, workorder-complete:29, recurring-invoices:22, recurring-jobs:30, process-scheduled-sends:205, seo-jobs:20, seo-audits:17, seo-reports:16, seo-gsc-sync:23. Only content-jobs:27 (unconditional compare) and email-leads:396-398 (logs and returns when unset) fail closed. booking-reminders:19-20 also reads the secret; its comment 'Light gate: allow Vercel Cron, or a matching CRON_SECRET if configured' confirms the same permissive intent. Severity kept at medium: production almost certainly has CRON_SECRET set (the crons demonstrably run), so the exposure is preview/new environments rather than prod today.

A24. Legacy INTERNAL_API_KEY fallback authenticates any caller as the oldest business on the platform

Severity: MEDIUM · **Effort:** minutes · **Domain:** Security & multi-tenancy

Evidence. `src/lib/api-auth.ts` — legacy fallback block: `const expected = process.env.INTERNAL_API_KEY; if (expected && legacyKey && legacyKey.length === expected.length) ... .from("businesses").select("id, user_id").order("created_at", { ascending: true }).limit(1).single()` returning scopes `["leads:read", "leads:write", "customers:read", "customers:write", "agent:access"]`; `src/app/api/report-sessions/route.ts` `checkKey`

Impact. One shared static secret grants lead and customer read/write to whichever business is the oldest row — today a real tenant with live data, and a target that changes silently if that row is deleted. Reachable through every `/api/v1` route (public in middleware). No tenant selection, no scoping, and revocation requires unsetting an env var and redeploying.

Recommendation. Delete the fallback. First check `vercel env ls` for `INTERNAL_API_KEY`; if any integration still uses it (the Telegram report-sessions routes do), mint that integration a real per-business key and migrate it, then remove the env var.

Verifier: Confirmed. `src/lib/api-auth.ts` contains the block verbatim: when the per-business key lookup misses, it length-checks then constant-time-compares the header against `process.env.INTERNAL_API_KEY`, and on match selects businesses ordered by `created_at` ascending limit 1, returning that business with `leads:read/write`, `customers:read/write` and `agent:access`. The related direct comparisons are also real (`src/app/api/report-sessions/route.ts` `checkKey` does `key === process.env.INTERNAL_API_KEY`). Two corrections: the block is inert unless `INTERNAL_API_KEY` is set (`if (expected && ...)`), so this is conditional rather than live-by-default; and the 'leaks the secret's length' claim is overstated — the response is an identical 401 either way, so length is only inferable via timing, which is not a meaningful escalation here.

A25. Public-endpoint rate limiting is in-memory per serverless instance, so the real limit is unbounded

Severity: MEDIUM · **Effort:** hours · **Domain:** Security & multi-tenancy

Evidence. src/lib/booking/public.ts — module-scope `const hits = new Map(...)` in `rateLimit()`;
src/app/api/f/[slug]/submit/route.ts `rateLimit(\ form:${slug}:${ip}`, 8, 60_000) ;`
src/app/api/f/[slug]/upload/route.ts `rateLimit(`formup:${slug}:${ip}`, 20, 60_000) ;`
src/app/api/audit/[slug]/submit/route.ts `rateLimit(`audit:${slug}:${ip}`, 5, 60_000)``

Impact. Each serverless instance keeps its own counter, so the effective ceiling is $N \times \text{limit}$ per minute and grows with concurrency. Every accepted submission calls `upsert_lead` and can trigger `notify_emails` from the business's verified Resend domain — a cheap way to poison a customer's pipeline, burn Resend sender reputation, or fill the public-form-uploads bucket 10MB at a time. The audit endpoint additionally queues `seo_audits` rows that draw on the Anthropic budget.

Recommendation. Move the counter to a shared store (Vercel Runtime Cache or Upstash) keyed identically, or put `/f/`, `/api/f/` and `/api/audit/` behind Vercel WAF rate-limit rules. At minimum add the per-form daily cap the code comment already assumes exists, enforced in the database so it holds regardless of instance count.

Verifier: Confirmed, and the code comments agree with the finding. `src/lib/booking/public.ts` declares `const hits = new Map<string, {n, reset}>()` at module scope with the comment 'Best-effort in-memory IP rate limiter. Fluid Compute reuses instances so this catches bursts; CAPTCHA + idempotency + per-day caps are the real guards' — note that the CAPTCHA and per-day caps that comment names as the real guards do not exist on the public-form paths. All three call sites check out: form submit 8/60s, form upload 20/60s, audit submit 5/60s. Mitigations the finding understates: the submit and audit routes do carry a `_hp` honeypot check that runs before the limiter, and required/invalid field validation runs server-side. Those raise the bar for naive bots but not for a deliberate attacker.

A26. Invoice numbers are minted with a read-modify-write counter in three places, and `invoices.number` has no unique constraint

Severity: MEDIUM · **Effort:** hours · **Domain:** Functionality & correctness

Evidence. `src/lib/actions/quotes.ts:145-153`; `src/lib/recurring/generate-invoice.ts:41-47`;
`src/app/api/portal/[token]/quote/[id]/accept-with-deposit/route.ts:24-33,99,145`; `src/lib/actions/invoices.ts:70-72`; no unique index found in `supabase/migrations/*.sql`

Impact. Concurrent creates mint the same number and both rows persist — nothing in the database rejects it. Two invoices sharing INV-0042 is a reconciliation/tax-record problem, and the failure is silent.

Recommendation. Add a service-role variant `next_invoice_number_admin(uuid)` (no `auth.uid()` check, EXECUTE to service_role only) and switch the two admin-client sites to it; switch `quotes.ts` to the existing RPC since it already runs as the user. Add `CREATE UNIQUE INDEX ON invoices (business_id, number)` so any regression fails loudly. Separately, guard the portal deposit route against double-submit.

Verifier: All three call sites verified. `src/lib/actions/quotes.ts:145-153` reads `invoice_next_number` then updates it — in a cookie-auth server action where the atomic RPC would work, so that one is a plain oversight. `src/lib/recurring/generate-invoice.ts:42-47` does the same and its own header comment (lines 5-6) explains why: the RPC checks `auth.uid()`, null for service-role. `src/app/api/portal/[token]/quote/[id]/accept-with-deposit/route.ts:24-33` defines `mintInvoiceNumber` with the same comment and calls it at line 99 and line 145. `src/lib/actions/invoices.ts:70-72` is the only atomic user. Grepped `supabase/migrations/*.sql` for any unique index touching invoices — none exists, so a collision inserts cleanly. Severity corrected down: at 42 invoices across 4 businesses with 2 active users, genuine write concurrency is close to nil; the realistic trigger is a portal double-click, and that route already produces duplicate invoices regardless of numbering, so numbering is not the primary defect there.

A27. `/api/cron/reminders` is hardcoded to a single business UUID — the feature is dead for every tenant except Crown Roofers

Severity: MEDIUM · **Effort:** hours · **Domain:** Functionality & correctness

Evidence. `src/app/api/cron/reminders/route.ts:14-17,68`; `vercel.json:6-9`; contrast `src/app/api/cron/invoice-reminders/route.ts:43-47`

Impact. Single-tenant code from the app's origin still running daily inside a multi-tenant SaaS. No other business's workers get a morning job email and no other business's customers get an appointment reminder; the Telegram digest goes to one hardcoded chat regardless of tenant. Nothing surfaces an error.

Recommendation. Either iterate businesses via `business_agent_installs` the way `invoice-reminders` does and resolve recipients per business (gating Telegram behind a per-business setting), or delete the route and its `vercel.json` entry if the capability is superseded.

Verifier: Verified. `src/app/api/cron/reminders/route.ts:14` hardcodes the fallback business UUID, `:15-16` bind one `TELEGRAM_BOT_TOKEN/CHAT_ID`, `:17` falls back to a Crown Roofers From address, and `getJobsForDate` at `:68` filters `.eq("business_id", BUSINESS_ID)` — the only `business_id` filter in the file. Scheduled daily at `0 20 * * *` in `vercel.json:6-9`. Also confirmed the contrast: `invoice-reminders/route.ts:43-47` correctly iterates `business_agent_installs`. Severity corrected down one notch: I found no agent-store card or UI surface advertising this cron to other tenants (grepped for a matching agent id and found none), so it is dead legacy code rather than a promised feature that silently fails — the damage is a missing capability, not a broken promise.

A28. Stripe webhook handles no refund, dispute, or failed-payment events — a refunded invoice stays marked paid forever

Severity: MEDIUM · **Effort:** days · **Domain:** Functionality & correctness

Evidence. `src/app/api/stripe/webhook/route.ts:31-45`; `src/lib/admin/audit.ts:23` (only refund reference in `src/`); no reversal path in `src/lib/stripe-payments.ts`

Impact. A refund issued from the Stripe dashboard leaves the payments row and the invoice's `amount_paid/status` untouched, so the invoice still reads paid and the revenue still counts in dashboard stats. A chargeback claws funds back with no in-app signal. Bank-delayed methods (BECS/SEPA, relevant for an

Australian merchant) that fail after `checkout.session.completed` are never reconciled, since only the `async_payment_succeeded` counterpart is handled.

Recommendation. Add `charge.refunded` (negative payment row keyed on the refund id for idempotency, then re-run the recompute), `charge.dispute.created` (flag and notify rather than auto-reverse), and `checkout.session.async_payment_failed`. Subscribe the live Connect endpoint to those events. Guard the recompute in `src/lib/stripe-payments.ts` so a reduced sum can produce statuses other than `paid/partial`.

Verifier: Verified. The switch at `src/app/api/stripe/webhook/route.ts:31-45` covers exactly `checkout.session.completed`, `checkout.session.async_payment_succeeded`, `payment_intent.succeeded` and `account.updated`, with a no-op default at :42-44. Grepped the whole of `src/` for refund/dispute handling: the only hit is an unrelated audit-action string literal "billing.refund" in `src/lib/admin/audit.ts:23` — no reversal logic exists. Severity medium is right: with zero Stripe payments ever taken, no refund has been missed yet, so this is a readiness gap rather than live data corruption. The finding's own note that the live endpoint subscribes to only 3 events comes from `CLAUDE.md`, not from code I could verify.

A29. Recurring billing mints invoice numbers with the read-modify-write race `CLAUDE.md` forbids, with no DB uniqueness backstop

Severity: MEDIUM · **Effort:** hours · **Domain:** Test coverage & quality

Evidence. `src/lib/recurring/generate-invoice.ts:41-47`; absence of any unique index on `invoices(business_id, number)` across `supabase/migrations/*.sql`; `CLAUDE.md` "Atomic number-mint RPCs"

Impact. Two recurring schedules maturing in the same cron tick, or a cron tick overlapping a user creating an invoice, can mint two invoices with the same number. Duplicate invoice numbers are an accounting/tax-compliance defect for the business being sold to.

Recommendation. Add a service-role-safe SECURITY DEFINER `next_invoice_number_admin(uuid)` doing a single atomic `UPDATE ... RETURNING`, call it from `generate-invoice.ts`, and add a unique index on `invoices(business_id, number)` as the backstop. Cover with a concurrency test in the style of `booking-db.test.ts:42-63` (once that suite is opt-in gated).

Verifier: Confirmed line-for-line. `src/lib/recurring/generate-invoice.ts:42-47` selects `invoice_next_number`, then separately updates it to `next + 1`, then formats the string — two round-trips, non-atomic. The header comment at :5-7 correctly explains the RPC can't be used from the service-role client. I grepped every file in `supabase/migrations` for a unique constraint or index touching `invoices+number` and found none, so there is no database backstop. `CLAUDE.md`'s Performance section does explicitly say to replace read-modify-write counter logic with the atomic RPCs. Caveat on severity: with 7 businesses, 42 invoices all-time and daily crons, the actual collision probability today is very low — this is a latent correctness defect, not an active fire.

A30. Zero tests on Stripe amount conversion and card surcharge

Severity: MEDIUM · **Effort:** hours · **Domain:** Test coverage & quality

Evidence. `src/lib/stripe.ts:36-48, 60-66, 91-100`; no `.test.ts` file in `src` references stripe

Impact. Stripe is live. The arithmetic that decides what a real card is charged has no regression net, so a future refactor of the face-amount-vs-surcharge split could go wrong silently. No current defect identified.

Recommendation. Add `src/lib/tests/stripe-amounts.test.ts`: round-trip `fromStripeAmount(toStripeAmount(x,c),c)` across AUD/JPY/KWD, `computeSurcharge` with disabled/null percent/null fixed, and `computeApplicationFeeAmount` returning an integer minor-unit value and 0 at 0%. Under an hour.

Verifier: The coverage claim is accurate: `src/lib/stripe.ts:36-48` (`toStripeAmount/fromStripeAmount` incl. the three-decimal branch at :39), :60-66 (`computeSurcharge`) and :91-100 (`computeApplicationFeeAmount`) are pure and dependency-free, and `grep -rln "stripe" src --include=*.test.ts` returns nothing across all 14 test files. Important correction to severity: I read the arithmetic and found no defect — the zero-decimal, three-decimal and two-decimal branches all look correct, and `computeSurcharge` already clamps negatives via `s > 0 ? ... : 0`. So this is missing coverage on a live-money path, not a demonstrated bug. Downgraded accordingly.

A31. Worker isolation — the product's one hard security boundary — has no test

Severity: MEDIUM · **Effort:** days · **Domain:** Test coverage & quality

Evidence. `src/lib/booking/tests/booking-db.test.ts:65-76` (anon-only); `src/lib/assistant/tests/scopes.test.ts:8-18`; `supabase/migrations/20260512000001_workers_can_see_job_customers.sql`

Impact. If a new business-data table ships without the `<table>_no_workers` policy — a mistake CLAUDE.md flags explicitly — a subcontractor-level user sees every customer, invoice and lead. Nothing in CI would notice. Also the hardest question to answer in a prospect's security review.

Recommendation. Add an opt-in-gated `src/lib/tests/rls-worker.test.ts`: temp business + worker member + member_profiles row, sign in as that user, assert 0 rows on invoices/customers/quotes/leads, only the assigned row on work_orders, and that the deliberate `workers_can_see_job_customers` path DOES return the assigned job's customer.

Verifier: The coverage claim checks out. The only RLS assertion in the repo is `booking-db.test.ts:65-76`, and it only tests that an ANONYMOUS client cannot read appointments — nothing tests an authenticated worker against invoices/customers/quotes/leads. `src/lib/assistant/tests/scopes.test.ts:13-18` does test that `assistantScopesForRole("worker")` is null, and its own comment (:8-10) explicitly says that guard exists because the tools bypass the DB layer — so the DB layer itself is indeed unverified. Severity note: this is absence of a test, not a demonstrated hole; the policies described in CLAUDE.md do exist in migrations. Downgraded from high to medium on that basis.

A32. 61 server-action files — the product's entire API surface — have zero tests

Severity: MEDIUM · **Effort:** days · **Domain:** Test coverage & quality

Evidence. 61 files in `src/lib/actions/`; 14 .test.ts files, none importing `src/lib/actions`; `src/lib/actions/invoices.ts` = 649 lines; `src/lib/tests/pg-filter.test.ts` exists

Impact. Every entry in CLAUDE.md's Known-traps list is a bug that reached production, was fixed, and now has nothing preventing its return across 61 files.

Recommendation. Don't chase coverage. Extract the recurring invariants into small pure helpers (a shared `uid()/coerceUuid` and a `num()` PostgreSQL coercion) and test those, plus a cheap grep-based test asserting no `.orC` call in `src/lib` takes an interpolated template literal — that single check would have caught PR #398 across all 17 sites.

Verifier: Counts verified: `ls src/lib/actions/*.ts` → 61; 14 test files total, all under `src/lib/{assistant,booking,content,mcp}/tests` and `src/lib/tests`, and none import from `src/lib/actions` (grep for 'lib/actions' across *.test.ts returns nothing). `src/lib/actions/invoices.ts` is 649 lines. `src/lib/tests/pg-filter.test.ts` does exist, so the `ilikeAcross` hardening is the one trap with a regression net. This is a true but general observation rather than a specific defect — I confirm the facts, not a bug.

A33. CLAUDE.md declares contracts/e-signature both shipped and "Not started"

Severity: MEDIUM · **Effort:** minutes · **Domain:** Documentation accuracy

Evidence. CLAUDE.md:411 vs CLAUDE.md:442; `src/components/layout/app-sidebar.tsx:40`; `src/lib/actions/contracts.ts` exists on branch

Impact. A developer or agent reading the "Not yet done / parked" list concludes contracts do not exist and may rebuild them, colliding with the live `contracts` table, storage bucket and MCP tools. The owner may also under-sell a shipped differentiator.

Recommendation. Delete the contracts bullet at CLAUDE.md:442 and change the heading at :411 to "(shipped, merged — PR #336)". Move the "Not yet done" list out of the append-only session log into a single rewritten "Current state" section.

Verifier: Verified both statements verbatim. CLAUDE.md:411 heading reads "### Customer contracts + NATIVE e-signature (shipped — PR #336, branch `feat/contracts`, NOT yet merged)" followed by ~8 bullets of implementation detail; CLAUDE.md:442 in "### Not yet done / parked" reads "**Customer contracts + e-signature:** NEXT FEATURE (requested)... Not started — design from scratch." The feature is real and on this branch: `src/lib/actions/contracts.ts` exists (last touched by 37b6ce7, not 05e0c85 as the finding cited — minor evidence error, conclusion unaffected), and `src/components/layout/app-sidebar.tsx:40` ships `{ label: "Contracts", href: "/contracts", icon: FileStack, plugin: "contracts" }`. Downgraded from high: this is pure doc drift with no runtime or customer-facing impact, and any reader who checks the sidebar sees contracts immediately.

A34. CLAUDE.md's sidebar section documents a nav structure and feature-flag mechanism that no longer exist

Severity: MEDIUM · **Effort:** minutes · **Domain:** Documentation accuracy

Evidence. CLAUDE.md:157-169 vs `src/components/layout/app-sidebar.tsx:17-24` (`NavItem.plugin`) and :27-79 (10 sections, 32 destinations)

Impact. The documented recipe for adding a conditional nav item is non-compilable, pushing a developer to hand-roll a second gating path parallel to the plugin registry — the exact ad-hoc flag sprawl the plugin system replaced.

Recommendation. Rewrite CLAUDE.md:155-170 to describe only plugin-id gating (tag the item with `plugin: "<id>"` matching `src/lib/plugins/registry.ts`). Delete the enumerated section list — point at the file instead, since any enumeration will drift.

Verifier: Verified precisely. CLAUDE.md:157 says "groups nav into 7 sections" and enumerates them; `src/components/layout/app-sidebar.tsx:27-79` defines 10 sections (Workspace, Sales, Service, Contacts, Catalog, Workforce, SEO, Content, Insights, Account) with 32 href entries, including Forms, Contracts, Recurring billing, Expenses, Bookings, Assets, Prospects, Onboarding, Inventory, Timesheets, Prospects, SEO Production and Content Studio — none documented. The gating claim is the sharper half: CLAUDE.md:169 instructs `NavItem.feature?: "quotingAgent"`, but `app-sidebar.tsx:17-24` defines `NavItem` with `plugin?: string` ("Owning plugin id (`src/lib/plugins/registry.ts`). Untagged = core, always shown.") — there is no `feature` key on `NavItem` at all. A developer following the documented recipe writes a property TypeScript rejects.

A35. Two-way SMS product is undocumented and has no MCP tool, violating CLAUDE.md's own hard rule

Severity: MEDIUM · **Effort:** hours · **Domain:** Documentation accuracy

Evidence. `src/lib/actions/sms.ts:28-29,33,79,92,106,180,191`; `src/app/api/sms/webhook/route.ts`; zero grep hits for `send_sms` in `src/lib/mcp/`; zero `clicksend` hits in `CLAUDE.md/docs/.env.local.example`

Impact. The AI assistant cannot satisfy the single most natural field-services voice request ("text the customer I'm running late"), undercutting the product's headline AI-first promise. Separately, a fresh deploy without `CLICKSEND_USERNAME/CLICKSEND_API_KEY` throws "ClickSend credentials not configured" with no documentation anywhere explaining the variable names.

Recommendation. Add `send_sms` and `list_conversations` MCP tools under a `messages:read|write` scope in `src/lib/mcp/tools/`, add a "### Messaging (SMS)" section to `CLAUDE.md` covering the ClickSend provider and inbound webhook, and add both ClickSend vars to `.env.local.example`.

Verifier: Verified on every limb. `src/lib/actions/sms.ts` exports `getConversations` (line 79), `getMessages` (92), `sendSms` (106), `markConversationRead` (180), `startConversation` (191), with ClickSend credentials read at lines 28-29 and the API call at line 33. UI exists at `src/app/(dashboard)/messages/page.tsx` and `src/components/messages/messages-client.tsx`; inbound at `src/app/api/sms/webhook/route.ts`. `grep -rn "send_sms|sendSms" src/lib/mcp/` returns nothing — no MCP tool, so the assistant genuinely cannot send an SMS. Case-insensitive grep for "clicksend" across `CLAUDE.md`, `docs/` and `.env.local.example` returns zero hits. This directly violates the standing rule at `CLAUDE.md:224` ("treat 'added a feature but not its MCP tool' as an incomplete change"). Downgraded from high only because it degrades a capability rather than breaking one — the SMS UI itself works when creds are set.

A36. .env.local.example documents 9 of 36 environment variables, including every payment and encryption key

Severity: MEDIUM · **Effort:** hours · **Domain:** Documentation accuracy

Evidence. .env.local.example (14 lines, 9 vars); 36 distinct process.env names across src/ and scripts/; no README.md at repo root

Impact. No single artifact answers "what must I set to run this?". Setup knowledge is scattered through 173 lines of chronological session log (Stripe at CLAUDE.md:387, ONBOARDING_SECRET_KEY at :427). Missing keys mostly fail quietly, and the knowledge is unrecoverable once the owner forgets it — a hard blocker on handing the codebase to a contractor.

Recommendation. Write a root README.md with Quick Start, and expand .env.local.example to cover the ~21 user-configurable vars grouped Required / Required-for-payments / Optional-per-feature, each with a one-line note on what breaks when absent. Add a CI check alongside scripts/check-bundle-size.mjs that fails when a process.env name is absent from the example.

Verifier: Counts verified. .env.local.example is 14 lines listing 9 active vars (Supabase x3, Resend x2, NEXT_PUBLIC_APP_URL, ANTHROPIC_API_KEY, OPENAI_API_KEY) plus commented Sentry. `grep -rhoE 'process\.env\. [A-Z0-9_]+ ' src/ scripts/ | sort -u` returns exactly 36 names. One caveat the finding omits: ~6 of those 36 are platform-provided and not user-configurable (NODE_ENV, NEXT_RUNTIME, VERCEL_ENV, VERCEL_URL, VERCEL_GIT_COMMIT_SHA, RUN_LIVE_API), so the true undocumented-and-configurable set is ~21, not 27 — the conclusion is unchanged. Every var the finding names as missing is genuinely missing: STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET, STRIPE_PLATFORM_FEE_PERCENT, APP_ENCRYPTION_KEY, ONBOARDING_SECRET_KEY, CRON_SECRET, GOOGLE_OAUTH_CLIENT_ID/SECRET, GITHUB_APP_*, CLICKSEND_*, RESEND_WEBHOOK_SECRET, KIREI_FROM_STRATEGY, KIREI_EMAIL_DOMAIN, INTERNAL_API_KEY, TELEGRAM_*, AGENT_BUSINESS_ID. `ls README.md` confirms no root README.

A37. docs/QA_CHECKLIST.md predates every revenue feature and covers none of them

Severity: MEDIUM · **Effort:** hours · **Domain:** Documentation accuracy

Evidence. docs/QA_CHECKLIST.md:3, sections 1-26, last commit 090f041 2026-05-08; zero grep hits for "stripe"

Impact. With 14 test files against ~88k lines of src/, this checklist is the de facto regression suite, and it never touches the code path that moves a real customer's money. Consistent with zero Stripe payments ever completing in production: nothing routinely exercises it, so the first paying customer is the first tester.

Recommendation. Add a "Section 27 — Getting paid" block (Connect onboarding, invoice email pay-link, portal card payment, deposit on quote accept, saved-card autopay charge, surcharge maths, receipt email) and run it end-to-end in live mode with a real card before selling to anyone. Backfill the other post-May features after.

Verifier: Verified and stronger than stated. The file is 447 lines, self-described at line 3 as "the canonical source", last committed 090f041 on 2026-05-08. Section headings run to "## 26. Cross-cutting / regression". A case-insensitive grep for "stripe" returns ZERO hits across the whole file — the live payment path is entirely untested by the only pre-release test artifact. The word "deposit" appears only at lines 128-133 and refers to the pre-Stripe progress-invoice modal, not card deposits. No coverage of contracts/e-signature, onboarding forms, public forms, bookings, expenses, inventory, timesheets, assets, prospects, SEO or Content Studio.

A38. The in-app help surface covers 14 of ~32 modules and says nothing about paying or getting paid

Severity: MEDIUM · **Effort:** days · **Domain:** Documentation accuracy

Evidence. src/components/help/help-client.tsx:16-130 (14 sections), lines 142/146/200 the only payment-adjacent text; src/components/layout/app-sidebar.tsx (32 destinations); src/lib/plugins/registry.ts (29 plugin ids); src/app/support/page.tsx (66 lines)

Impact. A trades-business owner cannot discover from inside the product that Kirei takes card payments, holds a card on file, auto-bills a retainer or gets a contract signed — the features that justify a subscription price. Every such question becomes a support email to the owner. Plausible contributing cause of the zero-Stripe-payments number, though not provable from the code.

Recommendation. Add help sections for the revenue path first ("Getting paid: connect Stripe, send a pay-link invoice, take a deposit, save a card"), then contracts, forms and bookings. Drive the section list off src/lib/plugins/registry.ts so a plugin without a help entry is a visible gap, and link the relevant section from each module's empty state.

Verifier: Verified. src/components/help/help-client.tsx defines a **sections** array with exactly 14 module entries (titles at lines 18,23,32,41,50,55,64,74,83,88,99,104,109,118: Dashboard, Invoices, Quotes, Customers, Products, Schedule, Recurring Jobs, Leads, Reports, Work Orders, Messages, Team, Agents, Settings). Against that, src/components/layout/app-sidebar.tsx has 32 href destinations (the finding said ~35 — minor overstatement) and src/lib/plugins/registry.ts defines 29 plugin ids (the finding said 31 — minor overstatement). The substantive claim holds exactly: grep for stripe|payment|deposit|contract|autopay across help-client.tsx returns only incidental hits at lines 142, 146 and 200 ("Invoice reminder cron chases payment", "Recurring Maintenance Contract", "Payment received → marked paid") — nothing explains Stripe, card payments, deposits, saved cards, contracts, e-signature, forms, bookings, expenses, inventory, timesheets, assets, prospects, SEO or Content Studio. src/app/support/page.tsx is 66 lines as stated.

A39. KPI strips on the plugin pages use a fixed **grid-cols-3** that never collapses on mobile

Severity: LOW · **Effort:** minutes · **Domain:** UI & UX consistency

Evidence. src/components/inventory/inventory-view.tsx:69; src/components/assets/assets-view.tsx:77; src/components/expenses/expenses-view.tsx:108; contrast src/components/leads/leads-client.tsx:261

Impact. At phone width each tile is roughly a third of the viewport, so multi-word labels wrap to several cramped lines and currency values crowd the column. These are the pages most likely to be opened from a phone on site.

Recommendation. Change the three to `grid-cols-2 lg:grid-cols-3`, or adopt StatTile as part of the primitives migration.

Verifier: Verified exactly as cited: `<div className="grid grid-cols-3 gap-3 mb-6">` with no responsive prefix at inventory-view.tsx:69, assets-view.tsx:77, expenses-view.tsx:108 (plus a fourth at inventory-view.tsx:114 inside a dialog). leads-client.tsx:261 does use `grid grid-cols-2 lg:grid-cols-4 gap-3`. Severity corrected from medium to low — three default-off plugin views, purely cosmetic, and the described overflow at 375px is inference since I did not render it.

A40. `.ch-table` gives every row a pointer cursor and hover highlight, but rows in the plugin tables aren't clickable

Severity: LOW · **Effort:** hours · **Domain:** UI & UX consistency

Evidence. src/app/globals.css:446-450; src/components/inventory/inventory-view.tsx:90; src/components/assets/assets-view.tsx:96; src/components/expenses/expenses-view.tsx:129; src/components/timesheets/timesheets-view.tsx:79

Impact. The row highlights and the cursor becomes a hand, promising navigation that never happens. Because the rule lives in the shared stylesheet it will mislead in every future `.ch-table` too.

Recommendation. Move `cursor: pointer` and the hover background onto an opt-in modifier such as `.ch-table.is-clickable tbody tr`, and apply that class only where a row navigates.

Verifier: Verified. src/app/globals.css:446-450 is `.ch-table tbody tr { transition: background 0.08s; cursor: pointer; }` and `.ch-table tbody tr:hover { background: hsl(var(--muted) / 0.6); }`. I read the row markup in all four cited views: inventory-view.tsx:90 `<tr key={p.id}>`, assets-view.tsx:96, expenses-view.tsx:129, timesheets-view.tsx:79 — none carries an onClick or a wrapping Link; only inline cell buttons act. Severity corrected from medium to low: a misleading affordance is a real but minor UX defect, and it is a one-line CSS change.

A41. `job_time_entries` has no (`business_id`, `started_at`) index

Severity: LOW · **Effort:** minutes · **Domain:** Performance & reliability

Evidence. src/lib/actions/timesheets.ts:44-47; supabase/migrations/20260418000001_account_site_portfolio.sql:313,326; 20260418000004_invoice_unbilled_work.sql:10; 20260710180000_timesheets.sql (whole file, 3 lines, no index).

Impact. The `/timesheets week` query has no supporting index and will seq-scan as time-entry volume grows. Time entries are the highest-cardinality row in a field-services product. No measurable impact at current data volume.

Recommendation. Add `CREATE INDEX IF NOT EXISTS idx_job_time_entries_business_started ON public.job_time_entries (business_id, started_at DESC);` plus `(business_id, user_id, started_at)`, mirroring 20260511020000_perf_indexes.sql — CLAUDE.md's 'mirror this index set' rule was skipped for this table.

Verifier: Index inventory is exactly as claimed: the only indexes on job_time_entries in the whole migrations tree are `jte_time_wo_idx ON (work_order_id)` (20260418000001_account_site_portfolio.sql:326) and `job_time_entries_invoice_id_idx ON (invoice_id)` (20260418000004_invoice_unbilled_work.sql:10). 20260710180000_timesheets.sql adds only `member_profiles.hourly_rate` and creates no index. The query at `src/lib/actions/timesheets.ts:45-46` filters `.eq("business_id", ...).gte("started_at", ...).lt("started_at", ...)`. Severity lowered to low: the claim is correct but the impact is entirely prospective — with 7 businesses and 42 invoices all-time, this table is small enough that the planner's choice is irrelevant today. It is worth doing because it costs one line, not because anything is slow now.

A42. prompts.test.ts fails on Windows because CRLF + non-ASCII in a .mjs breaks Vitest's transform

Severity: LOW · **Effort:** minutes · **Domain:** Test coverage & quality

Evidence. `prompts.test.ts:7` imports `../scripts/build-content-prompts.mjs`; reproduced `SyntaxError`; `scripts/build-content-prompts.mjs` CRLF + non-ASCII at offset 180; no `.gitattributes`; git config `core.autocrlf=true`

Impact. The guard that stops a stale `agent-prompts.generated.ts` shipping an old prompt never executes on the maintainer's machine, and a permanently-red local suite trains the developer to ignore red suites.

Recommendation. Add `.gitattributes` with `* text=auto eol=lf` (at minimum `*.mjs text eol=lf`) and `git add --renormalize .`. Do not remove the `.mjs` import — the drift check is the point of the file.

Verifier: Reproduced directly. `npx vitest run src/lib/content/__tests__/prompts.test.ts` in this worktree fails with `SyntaxError: Invalid or unexpected token` — 0 tests, whole suite fails to load. Confirmed `scripts/build-content-prompts.mjs` is 'UTF-8 Unicode text, with CRLF line terminators' with 65 CR bytes and first non-ASCII at byte offset 180, `core.autocrlf=true`, and no `.gitattributes` in the repo. The import is at `prompts.test.ts:7` and the drift assertion it guards is at :12-23. Severity note: CI is unaffected (LF checkout), and the failure is local-developer-experience rather than customer-facing — downgraded from medium to low, though it should be fixed first because it makes the whole local suite read as red.

A43. Per-customer payment-method gating is pure, trivially testable, and untested

Severity: LOW · **Effort:** minutes · **Domain:** Test coverage & quality

Evidence. `src/lib/payment-methods.ts:32-56` (esp. :40, :48, :53-55); consumer at `src/lib/recurring/generate-invoice.ts:108-110`; no matching test file

Impact. A future refactor of the null/empty-list semantics could surface a card button — and a regulated card surcharge — to a customer restricted to bank transfer, with no test to catch it. No current misbehaviour.

Recommendation. Add `src/lib/tests/payment-methods.test.ts` table-driving allowed = null | [] | ['cash'] | ['bank_transfer'] | ['card'] crossed with `stripeEnabled/hasBankDetails`. ~20 minutes.

Verifier: Coverage claim confirmed — `src/lib/payment-methods.ts` has no test file, and the cited behaviours are exactly as described: empty array collapses to null at :40 (`opts.allowed && opts.allowed.length ? new Set(...) : null`) and at :54, and cash is opt-in only at :48. It does gate the pay-by-card URL at `src/lib/recurring/generate-invoice.ts:108-110`. Two corrections that lower the severity: (a) the empty-array-means-all behaviour is not a lurking surprise — it is stated in the file's own header comment at :1-11 and in the param doc at :33 as intended semantics; (b) no defect was found, only absent coverage. Downgraded from medium to low.

A44. docs/MOBILE_PARITY_PLAN.md is stale and contradicts CLAUDE.md, while its update rule is still asserted as binding

Severity: LOW · **Effort:** hours · **Domain:** Documentation accuracy

Evidence. `docs/MOBILE_PARITY_PLAN.md:5,9-18`; `CLAUDE.md:7` and :531-534; git log last touch 8f4ffe7 2026-06-21

Impact. The only artifact describing what a mobile user can do is wrong in both directions, so it cannot answer a prospect's "does the app do X?". A visibly dead standing rule also devalues the other standing rules in `CLAUDE.md`.

Recommendation. Either refresh the matrix in one pass and enforce it with a PR-template checkbox, or retitle the doc "historical plan (May–June 2026), no longer maintained" and delete the policy at `CLAUDE.md:531-534`. A dead rule is worse than no rule.

Verifier: Verified. `docs/MOBILE_PARITY_PLAN.md:9-18` reads "## Where we are today (May 2026) — ## Mobile (Connected Hub) — currently a worker-only app ... ❌ Everything else"; `CLAUDE.md:7` states mobile "is no longer the worker-only 'Connected Hub' of earlier docs: it now has full feature parity with the web". The plan's line 5 does say "Update the matrix below in the same PR that changes web behaviour", repeated as numbered policy at `CLAUDE.md:531-534`. Last touch of the plan is 8f4ffe7, 2026-06-21, predating contracts, onboarding forms, form builder, plugins, SEO and the assistant rebuild. Downgraded to low: `CLAUDE.md:7` already explicitly retracts the stale doc by name ("of earlier docs"), so the misleading window is small, and no customer-facing surface depends on it.

A45. AGENTS.md is a stale sed-substituted fork of CLAUDE.md, now 41 lines behind

Severity: LOW · **Effort:** minutes · **Domain:** Documentation accuracy

Evidence. `AGENTS.md:3,10,207-209` (495 lines) vs `CLAUDE.md` (536 lines); git branch -a shows `claude/dreamy-robinson-9fcc52`

Impact. Two divergent architecture docs, one wrong about branch names and integration targets. An agent defaulting to `AGENTS.md` gets the June 2026 world and will not know the plugin system, assistant rebuild or Content Studio exist. Every `CLAUDE.md` edit silently widens the gap.

Recommendation. Replace AGENTS.md with a one-line pointer to CLAUDE.md. If a separate file is genuinely needed, generate it in CI rather than maintaining a hand-forked copy.

Verifier: Verified line-for-line. AGENTS.md is 495 lines vs CLAUDE.md's 536. AGENTS.md:3 reads "This file provides guidance to Codex (Codex.ai/code)"; AGENTS.md:10 references branch `Codex/dreamy-robinson-9fcc52` while `git branch -a` shows the real branch is `claude/dreamy-robinson-9fcc52`; AGENTS.md:207-209 reads "### MCP server — drive the app from Codex / Codex.ai" and "the **Codex.ai connector**" for an OAuth flow built specifically for claude.ai under `src/app/api/oauth/`. The tail of AGENTS.md ends at the mobile-parity policy, confirming the July 2026 plugin/SEO session log is absent. Downgraded to low: impact is confined to agents that happen to read AGENTS.md, and the fix is deleting one file.

A46. CLAUDE.md's append-only session-log structure generates the contradictions, and its top-level pointers are dead

Severity: LOW · **Effort:** hours · **Domain:** Documentation accuracy

Evidence. CLAUDE.md:10,12,13 vs missing SCOPE.md, docs/ contents, and docs/SEO_AGENCY_PLAN.md:3; session log starts CLAUDE.md:364 (173 of 536 lines)

Impact. A reader following the first 13 lines of the project's primary doc chases three files or statuses that do not exist, and burns context on 173 lines of historical narrative on every task. The append-only format is the mechanism behind the contracts and agent-surface contradictions confirmed above.

Recommendation. Delete the dead pointers at CLAUDE.md:10 and :12 and correct :13 to reflect the shipped status — these are minutes of work and verified. Treat the proposed CLAUDE.md/CHANGELOG split as an optional editorial improvement, not a defect fix.

Verifier: The factual half verifies cleanly: CLAUDE.md is 536 lines, the session log begins at line 364 ("## Session log (July 2026)"), giving 173 log lines (32%). `ls SCOPE.md` → no such file; `ls docs/` returns only MOBILE_PARITY_PLAN.md, QA_CHECKLIST.md, SEO_AGENCY_PLAN.md, so the docs/invoicer-scope-and-flow. {html,pdf} pointer at CLAUDE.md:12 is dead; and CLAUDE.md:13 calls the SEO plan the "NEXT BIG INITIATIVE (July 2026)" while docs/SEO_AGENCY_PLAN.md:3 reads "**Status: P0-P4 SHIPPED** (July 2026, PRs #359–#379)". Three of four pointers in the first 13 lines are wrong. The structural half of the finding ("split CLAUDE.md, append-only format guarantees decay") is a reasonable editorial argument but is opinion, not a verified defect — I confirm this finding on the dead-pointer evidence only, and downgrade severity accordingly.

Appendix B — Competitive analysis (raw)

Competitors

ServiceM8

- **Positioning.** The default for Australian solo tradies and 1-5 person crews. Job-first, photo-driven, iOS-only. Owns the AU small-trades mindshare and is the product Kirei most directly clones.
- **Pricing.** AUD flat per business, NOT per user — Free \$0 (30 jobs/mo, 1 user), Starter \$29 (50 jobs), Growing \$79 (150 jobs), Premium \$149 (500 jobs), Premium Plus \$349 (1500+ jobs). Unlimited users on every paid plan. Card processing 1.49–2.10% + 30c, SMS 10c overage. 14-day trial.
- **Leads with.** Job scheduling and dispatch, quotes/invoices from the phone, job photos and forms/certificates, online booking, Xero/MYOB/QuickBooks sync, asset management (Growing+), job costing and markup billing (Premium+), 'AI Smart Helpers' with unlimited daily uses.
- **Common complaints.** No Android app at all — repeatedly cited as the single biggest dealbreaker for AU tradies who buy Android hardware ('with Android we lose everything'). Clunky client input (only two contacts visible per job), clunky/slow invoicing flow, app lags and loads slowly in the field, weak inventory module, limited customisation, customer portal is basic. Job caps force upgrades regardless of team size.

Tradify

- **Positioning.** NZ/AU quoting-and-job-management for growing small trades businesses. Simpler than simPRO, more structured than ServiceM8. Strong Xero story.
- **Pricing.** Per user per month ex GST (AU page): Lite ~\$48, Pro ~\$52, Plus ~\$62. Aggressive discounting is routine (EOFY 2026 promo: 50% off first 6 months, code AUEOFY26). Add-ons: Instant Website \$19/mo, SMS quotes/invoices and reminders 20c/message. 14-day trial.
- **Leads with.** Two-way Xero sync (invoices, contacts, items, payments), unlimited jobs, quoting, scheduling, timesheets, GPS tracking, mobile app for both platforms.
- **Common complaints.** Per-user pricing gets expensive fast — a 5-person crew is ~\$260/mo before add-ons, and the price is the most common objection. Cloud-first with limited offline capability: the app slows or fails to sync in rural areas, basements and new subdivisions with poor reception. Weaker job costing and margin visibility than Fergus, which is the standard reason people leave.

simPRO

- **Positioning.** Enterprise/commercial contracting. Project management, complex job costing, multi-crew, inventory, maintenance contracts. Explicitly not for small operators.
- **Pricing.** Quote-only — no public price. Reported ~\$70/user/month at the low end; 10 users typically \$700–1200/mo. Implementation/training/migration adds \$3,000–10,000 for SMBs, \$20,000+ for enterprise. AU customers report 3- or 5-year lock-in contracts with ~8%/yr escalation (CPI+5%), or ~12%/yr on shorter terms.
- **Leads with.** Job costing and profitability, project management, inventory and stock control, maintenance/service contracts, deep accounting integration, enterprise reporting.

- **Common complaints.** 'Far too complex' — producing desired results feels impossible even after training. 2–3 month implementations, usually requiring a paid partner. Features promised in the sales process quietly fall away during implementation. Support: long waits, delayed tickets, uneven follow-through. Reviewers explicitly advise small businesses without dedicated admin staff not to buy it, and point them to Fergus or Tradify instead. Lock-in contracts with automatic escalation are a recurring grievance.

Jobber

- **Positioning.** North American market leader for home-service SMBs (landscaping, cleaning, HVAC, plumbing). Polished, consumer-grade, heavy marketing spend. Weak AU presence.
- **Pricing.** USD. Individual Core \$39/user, Connect \$119, Grow \$199. Team plans: Connect Teams \$169 (5 users), Grow Teams \$349 (10 users), Plus \$599 (15 users). Extra users \$29/mo. Annual billing saves up to ~40% (Core drops \$39→\$29).
- **Leads with.** Scheduling and dispatch, route optimisation, client hub, quoting/invoicing, online booking, automated reminders and follow-ups, marketing add-ons, QuickBooks sync.
- **Common complaints.** Real cost is far above headline: a 6-person team with the add-ons most businesses actually need runs \$329–449/mo. QuickBooks sync, automated texting, GPS tracking and professional proposals are gated behind higher tiers. Feature gating between Core/Connect/Grow frustrates users who discover the thing they need is one tier up. Minimal AU localisation (GST, ABN, AU payment rails).

Housecall Pro

- **Positioning.** US residential home services with the strongest consumer-facing experience — customer notifications, booking, review generation. Sales-led growth.
- **Pricing.** USD. Basic \$59/mo annual (\$79 monthly, 1 user), Essentials \$149/mo annual (\$189 monthly, up to 5 users), MAX \$299/mo annual (\$329 monthly, ~8 users). Additional users \$35/mo. 14-day trial.
- **Leads with.** Consumer-grade booking and notifications, dispatch, invoicing and payments, review generation, marketing automation, QuickBooks integration.
- **Common complaints.** Price increases at renewal with little warning is the most-repeated complaint. Support quality has declined — Trustpilot ~3.2/5, with slow response times and unhelpful chatbots called out repeatedly. Basic plan is missing QuickBooks sync and GPS tracking, so most teams are forced to Essentials or MAX. Fully-equipped 6-person cost lands \$369–468/mo. No AU market presence.

GoHighLevel

- **Positioning.** White-label marketing/CRM platform sold to agencies, not to trades businesses directly. The 'resell it as your own software' play. Kirei borrows its CRM/forms/automation half.
- **Pricing.** USD. Starter \$97/mo (3 sub-accounts), Unlimited \$297/mo (unlimited sub-accounts, rebilling at cost), Agency Pro \$497/mo (SaaS Mode — set your own price, keep the margin, auto-rebill SMS/email). Annual billing ~17% off (\$970 / \$2,970 / \$4,970). Usage (SMS, email, calls) billed on top via wallet.
- **Leads with.** Funnels and landing pages, form/survey builder, pipelines, SMS and email automation, calendars/booking, reputation management, white-label SaaS Mode with rebilling.
- **Common complaints.** Not a field-services product — no jobs, no scheduling for crews, no job costing, no accounting sync. Steep learning curve and a reputation for buggy releases. Usage-based wallet charges on top of the subscription make costs unpredictable. Sold hard by affiliates, which colours much of the online review corpus.

AroFlo

- **Positioning.** Australian-built, mid-market field service and job management. Sits between Tradify and simPRO. Strong AU compliance and payroll/award story.
- **Pricing.** No public pricing — the pricing page is a 'Request Pricing' lead-capture form (verified by fetching aroflo.com/pricing). Third-party aggregators report a ~\$39/mo entry point, but this is unverified and AroFlo does not publish per-user rates.
- **Leads with.** Quoting and job management, timesheets and scheduling, accounting and payments integrations, AU award/payroll handling, compliance forms, asset and maintenance management.
- **Common complaints.** Opaque quote-only pricing is itself a friction point — buyers can't self-serve compare. Positioned as more complex than Tradify/Fergus, which pushes small operators away. (I found no reliable body of specific user complaints in the sources I read — treat this row as lower-confidence than the others.)

Fergus

- **Positioning.** NZ-built, margin-and-job-costing focused. The product AU/NZ tradies move to from Tradify when they want to see profit per job. Actively markets against Tradify.
- **Pricing.** Reported from ~\$44 per user per month, tiered by business size and features. (Sourced from comparison coverage rather than a fetched Fergus pricing page — medium confidence.)
- **Leads with.** Job costing and margin visibility per job, financial reporting, scheduling and dispatch, quoting, accounting sync. Runs a dedicated 'Tradify alternatives' campaign targeting Tradify's costing weakness.
- **Common complaints.** Value depends heavily on setup quality — 'when it starts to feel clunky it usually means the setup, workflows or training need attention', i.e. it needs configuration effort most solo tradies won't invest. Per-user pricing has the same scaling problem as Tradify. Heavier than ServiceM8 for a one-person operation.

Where Kirei is ahead

- First-party MCP server with full OAuth 2.1 — genuinely nobody else has this. `src/app/api/mcp/route.ts` exposes ~199 tools via `createMcpHandler + withMcpAuth`, and `src/app/api/oauth/{register,authorize,token}` is a real authorization server with PKCE S256 and a consent screen that mints a revocable `inv_*` key as the access token. That means a tradie adds Kirei as a claude.ai custom connector and runs their business by talking to Claude. The only comparable thing in the market is a third-party community wrapper around the Housecall Pro REST API listed on mcpmarket.com — an unofficial shim, not a vendor-built, OAuth-gated, parity-complete surface. This is the one thing in the product that is defensibly ahead, and it is worth roughly nothing until `ANTHROPIC_API_KEY` is topped up.
- One tool registry serves both MCP and the in-app assistant. `src/lib/mcp/register-tools.ts` takes a `ToolFn` rather than a server; `/api/mcp` adapts it onto the real MCP server and `src/lib/mcp/collect.ts` collects the identical tools as plain data for `/api/assistant`. Adding a capability lights it up in Claude Desktop, claude.ai and the in-app agent simultaneously. Every competitor's AI is a bolted-on feature (ServiceM8's 'AI Smart Helpers') against a fixed feature list; Kirei's AI surface is the app.
- Correct agentic memory. `CLAUDE.md` documents that the wire format is `Anthropic.MessageParam[]` with `tool_use/tool_result` blocks preserved, and `assistant_messages.content` is JSONB for that reason. This is the difference between an agent that remembers it looked up Sarah's ID five turns ago and a chatbot. It is a real engineering asset and it is invisible to buyers — do not try to sell it, just let it make the demo work.

- Undo on agent actions — `src/lib/assistant/undo.ts` is a deliberate allow-list reusing the `cleanup_runs` `change_log` shape, excluding creates, sends and charges. Nobody else lets an AI mutate business data at all, let alone reverse it. This is the answer to the objection every tradie will raise ('what if it stuffs up my invoices').
- Native e-signature contracts at zero marginal cost. Per `CLAUDE.md` the Dropbox Sign integration was ripped out and replaced with a first-party flow: portal signing page, typed-or-drawn signature, IP/UA/consent audit into a `jsonb` column, and a `@react-pdf`-generated signed PDF with a certificate page. Competitors either don't do contracts or make you pay a third party. For a roofer doing \$327k in open quotes this is a genuine differentiator.
- Stripe Connect done properly — direct charges on the business's own connected account with `application_fee` to the platform, card-on-file autopay via off-session `PaymentIntents` (`src/lib/stripe-charge.ts`), an idempotent shared recorder (`src/lib/stripe-payments.ts`) used by both webhook and inline paths, per-customer allowed payment methods, quote deposits, and card surcharge with the invoice credited the `FACE` amount via `kirei_amount` metadata. This is more sophisticated than what ServiceM8 exposes. It has also never processed a single real payment, which means it is architecture, not evidence.
- GHL-class CRM in a trades product. Public form builder with hosted `/f/[slug]` plus `iframe /embed/[slug]` and lead creation through `upsert_lead`, client onboarding forms with AES-256-GCM secure fields, lead dedup enforced at the database level via a generated `identity_key` column and unique index. ServiceM8 and Tradify have none of this; getting it from GoHighLevel costs USD \$97–497/mo on top of your job software. This is the strongest non-AI story Kirei has and it is currently buried.
- Worker isolation enforced in the database, not the UI. `SECURITY DEFINER` helpers (`is_business_worker`, `my_member_profile_ids`) back RLS policies on every sensitive table, so a worker cannot see invoices, customers or margins even through a compromised client. Most competitors gate this in application code. Worth one line on a pricing page ('your subbies see their jobs, nothing else') and no more.
- Multi-business from day one — `businesses.user_id` plus `business_members`, `business_id` on every row, cookie-driven switcher. A tradie running two entities, or a bookkeeper serving several, is a first-class case rather than a workaround.

Where Kirei is behind

- No accounting integration whatsoever. I grepped `src/` for `xero|myob|quickbooks` and the ONLY hit is a prompt string inside `src/app/api/cron/email-leads/route.ts` line 150 listing 'Stripe/PayPal/Xero emails' as transactional mail to ignore. There is no connector, no OAuth app, no sync. Every single competitor leads with two-way Xero/MYOB sync. In Australia this is not a feature gap, it is a disqualification — the bookkeeper decides, and the bookkeeper will not accept re-keying invoices. Nothing else on this list matters until Xero exists.
- No way to charge for Kirei itself. I found no subscription, plan, trial or billing-period code anywhere in `src/lib/actions` or the settings pages. Stripe Connect lets your customers take money; there is no mechanism for you to take money. The product cannot be sold today in the literal sense. This outranks every feature request in the repo.
- No offline capability in the mobile app. `mobile/package.json` has `async-storage` and `react-native-mmkv` but no write queue, no sync reconciliation, and `grep` for 'offline' across `mobile/src` returns only `notifications.ts`. Tradify is actively criticised in AU reviews for exactly this — apps failing in basements, rural blocks and new subdivisions. A roofer standing on a roof in a Telstra blackspot who loses a job photo will churn that week, and will tell other roofers.

- No GPS or location tracking. `mobile/package.json` has no `expo-location`. Tradify ships GPS on all plans; Jobber and Housecall Pro gate it behind mid tiers but have it. Any business with 3+ vehicles asks about this on the first call.
- Android is not shipped. `CLAUDE.md` states the `.aab` is built but the first Play release requires a manual Console upload that hasn't happened. ServiceM8's most-cited weakness in AU reviews is being iOS-only — that is Kirei's single clearest wedge against the market leader, and it is sitting unexercised behind a form upload. This is hours of work for the highest-leverage competitive claim available.
- The AI differentiator is dead in production. `ANTHROPIC_API_KEY` returns 401, so the assistant, quoting agent, SEO pipeline and content studio all fail. The entire ahead-of-market position is currently a broken demo. If a prospect clicks Assistant in a trial, the product looks abandoned.
- No error monitoring. `SENTRY_DSN` is unset, so `src/instrumentation.ts` is inert by design. With paying customers you will learn about outages from angry phone calls. Housecall Pro's support reputation is a competitive liability worth attacking — you cannot attack it while flying blind.
- One Supabase project for local, preview and production. `CLAUDE.md` admits this openly: local dev reads and writes production data and every migration is a production change. With 7 businesses that is a bad night. With 70 paying businesses and one real roofer's \$17k in receivables, it is an existential event. Competitors at this price point have staging.
- 14 test files against ~88,000 lines of `src/`. The payment path — the one that moves other people's money and has never once run in anger — is the least defensible place in the codebase to have thin coverage.
- No job costing or margin-per-job as a headline surface. `get_job_costing` and `get_work_order_financials` exist as MCP tools, but this is the specific reason AU/NZ tradies leave Tradify for Fergus and the reason they tolerate simPRO's complexity. It needs to be a page a business owner opens weekly, not a tool an AI can call.
- No route optimisation. Jobber leads with it; anyone running multiple crews across a metro area asks. Lower priority than the rest of this list but it will come up in sales calls.
- No migration path in. There is no importer for ServiceM8, Tradify or a customer CSV. Switching costs are the entire moat in this category — a tradie with 400 customers and three years of job history will not retype them, and asking an AI to do it is not an answer you can put in a trial flow.

Not worth maintaining

- The entire SEO Production plugin (`src/app/(dashboard)/seo`, `src/lib/seo/*`, 7 tables, 13 markdown agents compiled by `scripts/build-agent-prompts.mjs`, the `seo_jobs` cron runner, the live agent terminal, budget caps, Opportunity Scout, and publish gateways for Git/GitHub/WordPress/Sanity/Payload/REST/GraphQL, plus the GSC OAuth connector and the `kireihq` GitHub App). This is the largest single body of code in the repo serving a market — SEO agencies — that is not trades, has zero paying users, and shares almost nothing with the job/invoice core. It also carries live operational burden: a Google OAuth app in Testing mode with 7-day token expiry, an `APP_ENCRYPTION_KEY` that can never be rotated, and per-run Anthropic spend. Cut it. If the SEO vertical is a real future business it is a separate product with a separate repo and a separate landing page.
- Content Studio (`src/app/(dashboard)/content`, per-client brands, topic scout, angle-to-platform fan-out, campaigns and calendar — PR #403). Same argument as SEO and newer, so less sunk cost. A roofer does not want a social content pipeline. Delete before it accretes more surface.
- Prospects and outbound (`src/app/(dashboard)/prospects`, `import_prospects`, `email_prospects`, `bulk_update_prospects`, `convert_prospect`). This is cold-outreach tooling — a GoHighLevel agency feature. Your trades customer has a full pipeline and needs help closing quotes, not buying lists. It also drags you toward spam-compliance obligations you do not want.

- The agency industry preset and the multi-vertical ambition in docs/SEO_AGENCY_PLAN.md. The plugin/preset machinery itself (src/lib/plugins) is fine and cheap — keep it as the on/off mechanism. Kill the pretence that Kirei serves agencies. Two verticals at 2 active users is how you end up with neither.
- The Quoting Agent as a separate surface — its own onboarding wizard, its own settings page, its own 12-iteration endpoint at /api/quoting-agent, its own quoting_agent_knowledge table, its own conditional sidebar item. The main assistant already has create_quote and the full tool registry. This is a second AI implementation with a second prompt-caching strategy and a second memory contract to keep in sync. Fold the pricing knowledge bank into the main assistant's context and delete the surface. Keep the knowledge bank table — per-business pricing memory is genuinely valuable.
- One of the two form builders. /onboarding-forms and /forms already share the OnboardingField engine and presets.ts, but they are two list pages, two builders, two submission viewers, two sets of MCP tools, two storage buckets and two mental models. Public lead-capture is the one that acquires customers. Merge onboarding into it as a delivery mode ('send privately to a customer' vs 'publish publicly') and delete the duplicate UI.
- The four field-services micro-plugins — expenses, inventory, timesheets, assets. Each is a table, a page, a set of actions and MCP tools. ServiceM8 users complain its inventory module is weak and mostly do not use it; simPRO's depth here is precisely what makes it unusable for small operators. Keep timesheets if the roofer actually uses them (check the DB before cutting). Ship the other three as stubs or remove them.
- Bookings (appointment types, booking resources, booking forms, exceptions, working hours — a full scheduling-link subsystem). This is Calendly for trades. Real, but it is a fifth priority behind Xero, billing, offline and Android, and it is carrying ~10 MCP tools and a settings surface right now.
- The MCP OAuth authorization server (src/app/api/oauth/*, oauth_clients, oauth_codes, the .well-known rewrites in next.config.ts). Painful to say because it is the best engineering in the repo — but it exists to serve claude.ai custom connectors, which no trades customer will configure. Header-key auth via Settings → API covers Claude Code and your own demos. Keep the code, stop treating it as a supported feature with a support burden. Revisit when a customer asks.
- Smart Organise as a per-entity feature on eight list pages (src/lib/actions/cleanup.ts, CleanupButton in every PageHeader, the cleanup_runs audit table). The undo machinery is load-bearing for the assistant — keep that. The eight deterministic proposers and eight buttons are polish on a data-quality problem that businesses with 42 lifetime invoices do not have yet.

Positioning

Position Kirei as: "ServiceM8 for Android, that you can run by talking to it." Australian trades businesses with 2–10 people. Not solo tradies (ServiceM8 Free/\$29 owns that and you cannot win a price war with a free tier), not commercial contractors (simPRO owns it and you have no project management), and emphatically not agencies.

The wedge, in priority order, is three specific competitor weaknesses you can attack with things you either have or can build in weeks:

1. ServiceM8 has no Android app, and AU reviews show this is its most-cited failure — tradies buy Android hardware. You have an .aab built and sitting unreleased. Shipping it converts your single best competitive claim from theoretical to true. Do this before anything else on the marketing side.
2. Tradify, Fergus, Jobber, Housecall Pro and simPRO all charge per user. That is the loudest complaint in every review corpus I read. Charge per business with unlimited users, like ServiceM8 does. Every apprentice a customer hires makes your pricing look better and your competitor's look worse.
3. ServiceM8 and Tradify have no CRM, no lead pipeline, no form builder, no automation. Getting that means adding GoHighLevel at USD \$97–497/mo. You already have leads with database-level dedup, public forms with embeds, onboarding forms, contracts with native e-signature, and recurring billing. That is a genuine two-tools-in-one

story and it is currently invisible.

Lead the marketing with jobs, quotes, invoices, photos and Android — the boring things buyers actually search for. The AI is the closer, not the opener: a 90-second demo where you say "quote the Henderson roof, three days, two guys, send it" and it happens. Tradies do not buy "AI-powered"; they buy "I did my quoting in the ute in four minutes."

Three things gate this, and none are optional. Xero sync — no accounting integration exists in the codebase at all, and in Australia the bookkeeper has veto. Subscription billing for Kirei itself — there is none, so there is currently no mechanism to be paid. A working ANTHROPIC_API_KEY — the entire differentiator 401s in production today. Until those three land you do not have a product to position, you have a codebase.

The honest read: you have built roughly three products (field services, an SEO agency platform, a content studio) and finished none of them commercially, while the one real customer — a roofer with \$17k outstanding and 98 jobs — depends on the least glamorous third. Cut to that third. The MCP/assistant work is real and ahead of the market, but it is a reason to choose Kirei over ServiceM8 after the buyer has already accepted that Kirei does the basics. It is not a reason to try it.

Pricing

Charge flat per business in AUD ex GST, unlimited users, no job caps. This deliberately copies ServiceM8's structure (the one AU tradies like) while removing its two irritants — job caps and iOS-only — and attacks the per-user model every other competitor uses.

SOLO — A\$79/mo. 1–2 users, unlimited jobs, quotes, invoices, photos, customer portal, Xero sync, mobile (iOS + Android), AI assistant with a fair-use cap (~300 messages/mo).

CREW — A\$149/mo. Up to 10 users, plus lead pipeline and public forms, contracts with e-signature, recurring billing and card-on-file autopay, job costing, SMS, unlimited AI.

BUSINESS — A\$299/mo. Unlimited users, multi-business, API and MCP access, priority support, custom email templates and branding.

Why these numbers, against what competitors actually charge:

A\$149 for a 10-person crew is the whole argument. Tradify Pro at ~\$52/user is ~\$520/mo for the same team. Fergus at ~\$44/user is ~\$440. Jobber Grow Teams is USD \$349 for 10 (~A\$530). simPRO is ~\$70/user plus \$3–10k implementation and a multi-year contract with ~8%/yr escalation. You are 3–4x cheaper at team scale and that is a sentence a tradie repeats to another tradie. Below about 4 users the per-user products are cheaper than you — accept that, it is why Solo exists and why you don't chase the 1-person market.

A\$79 for Solo is deliberately equal to ServiceM8 Growing, and Growing caps at 150 jobs/mo. You match the price, remove the cap, and add Android. Do NOT build a free tier or a \$29 tier. ServiceM8 has a genuinely good free plan backed by a decade of scale; competing there acquires users who will never pay and generates support load you cannot absorb at 13 total users. Run a 14-day trial like everyone else does, card required at the end.

A\$299 Business is priced against Housecall Pro MAX (USD \$299) and GoHighLevel Unlimited (USD \$297) — the point of the tier is that it exists, so that Crew reads as the sensible middle. Most customers should land on Crew.

Keep the Stripe platform fee at 2% (STRIPE_PLATFORM_FEE_PERCENT already defaults to 2). It is real revenue, it scales with customer success, and it is invisible in the sticker price. Be careful with card surcharge as a default — surcharging is regulated in AU and the compliance warning already in the Settings UI is correct; leave it off by default.

Annual billing at 2 months free (~17% off) matching GoHighLevel's discount shape, once you have a churn number worth protecting. Not before — annual prepay on a product with no error monitoring and a shared prod/dev database is a refund liability.

One caution on the AI economics. Unlimited assistant usage on Crew at A\$149/mo is unmodelled. A tradie who leaves a long-running agent loop going can burn real Anthropic spend, and every tool call ships a full business snapshot. Instrument per-business token cost from the first paying customer, and reuse the seo_monthly_budget_cents cap pattern you already built rather than discovering the problem on a card statement.

Appendix C — Design analysis (raw)

Assessment

Kirei is not amateur, but it is *four design systems wearing one trench coat*, and the seams are visible on any screen a paying customer would see.

The good news first: the Connected Hub token layer in `src/app/globals.css` (lines 134-179) is genuinely well-judged. The warm off-white canvas `60 9% 98%`, white cards, warm-grey borders `50 17% 88%` and deep teal `191 38% 36%` are a mature, calm, unfashionable-in-a-good-way palette that reads as trustworthy to a trades audience. The `--radius: 0.625rem` decision and the Plus Jakarta tracking corrections (`globals.css:118-125`, with a comment explaining *why* geometric sans needs tighter tracking) show real craft. And the just-shipped Leads workspace is the quality bar the brief claims: `src/components/leads/lead-card.tsx:3-11` contains a design rationale in prose — "the old card filled its whole surface with one of five stage colours, so a full board was five columns of saturated blocks and nothing stood out" — and the fix (neutral card, 3px stage rail) is exactly right. Someone on this project can design.

The problem is that this reasoning was applied to Leads and nowhere else. Concretely, the app has **four independent status-colour vocabularies** in simultaneous use:

1. `.ch-pill` (`globals.css:455-493`) — soft warm tints, leading dot, sentence case
2. `KireiPill` (`src/components/ui/kirei/pill.tsx:44-51`) — Tailwind `bg-emerald-100`, 10px ALL-CAPS, no dot
3. `STAGES` (`src/components/leads/lead-shared.ts:23-29`) — a third set of `bg-blue-50` / `text-blue-700` tints
4. `STATUS_GRADIENT` gradient tiles keyed on invoice status (`invoices-client.tsx:183`)

So "Paid" is a warm mint pill on `/messages`, a saturated ALL-CAPS emerald pill on `/invoices`, and an emerald gradient square on the same row. A user cannot learn one visual language.

Two things here are outright bugs, not taste. First, `.ch-pill` has no dark-mode variant at all — the backgrounds are hardcoded `hsl(150 50% 95%)` etc. with no `.dark` override anywhere in `globals.css`. In dark mode those pills render as near-white blocks. They appear in `worker-dashboard.tsx`, `messages-client.tsx` and `briefing-bell.tsx` — the worker dashboard is the crew-facing screen used on a phone outdoors, which is precisely where dark mode gets used. `.ch-stat-delta.up/.down` (`globals.css:412-413`) has the same defect. Second, `useIsDark()` — duplicated verbatim in `gradient-tile.tsx:25-37`, `stat-tile.tsx:25-37` and `avatar.tsx:26-38` — initialises to `false` and only corrects inside `useEffect`. That means SSR and first client paint *always* render the light gradient palette, then flip. In dark mode every gradient tile, every KPI tile and every avatar in the app visibly flashes pale teal on load. Ironically the fix already exists and is unused: `gradientVars()` at `gradient-tokens.ts:63-74` emits CSS custom properties for exactly this purpose.

On saturated colour, the brief's instinct is correct and it hasn't been fixed. `KireiAvatar` hashes every name into a nine-gradient pool including rose, coral, amber, violet, dusk and ocean (`avatar.tsx:12`), all at full saturation (`gradient-tokens.ts:12-29`). Open `/customers` with 50 rows and you get a rainbow — and because colour is doing nothing, the user's pattern-matching instinct is being burned for free. The dashboard compounds it: four quick actions get four different saturated gradients (`dashboard-client.tsx:150-153`) for four equally-weighted, non-semantic buttons. Meanwhile the actually-urgent signal — an overdue invoice — competes for attention against all of it.

On hierarchy and density, the invoice row is the clearest miss. `invoices-client.tsx:170-200` renders a 40px saturated gradient tile (no information), then the invoice number in muted 12px mono, then the customer name at `text-sm font-semibold` — and then the *money*, the single most important field on the screen, also at `text-sm font-semibold`. Stripe would render that number largest and boldest. Worse, this row is a table pretending not to be one: it has fixed-width header spans (`w-24`, `w-28`, `w-32`) and `divide-y` rows built from divs, so it forfeits `.ch-table`'s tabular-nums, mono refs, and semantics — while seven other views (`expenses-view`, `inventory-view`, `assets-view`, `timesheets-view`, `prospects-view`, `analytics-client`, `seo-performance`) use the real `.ch-table`. Two answers to the same question, and the newer one is the weaker one. The `flex-wrap` on that row also means it reflows to ragged heights at tablet widths, destroying column scanning.

Smaller drift worth naming: `.ch-stat` and `.ch-table-wrap` hardcode `border-radius: 14px` (globals.css:382, 420) instead of `var(--radius-xl)`, so they silently desync from the token CLAUDE.md documents. Two competing KPI treatments coexist — plain white `.ch-stat` on /analytics and worker, tinted-gradient `StatTile` on the other eleven surfaces. `StatTile`'s icon chip hardcodes `rgba(255,255,255,0.55)` (`stat-tile.tsx:53`), which is a glary white chip on a `#143536` tile in dark mode. And `layout.tsx:9` sets `themeColor: "#2563eb"` — bright blue — so the installed PWA's browser chrome and Android status bar are blue while the entire app is deep teal. That is the very first colour a customer sees on a phone, and it's from a design system the app abandoned.

Impression on a non-design-literate trades customer: competent but restless. Nothing looks broken, but the app never settles into a single voice, and the rainbow avatars plus decorative gradient tiles read slightly "app-builder template" rather than "tool my business runs on". Against ServiceM8 and Tradify — whose entire competitive claim is calm simplicity — visual noise is a live commercial liability, not a polish item. The single highest-leverage move is not new design work: it is deleting three of the four status systems and desaturating the decorative colour, applying the reasoning already written down in `lead-card.tsx` to the other twenty screens.

What competitors do well

ServiceM8

- **Does well.** Mobile-first with near-zero UI chrome. Reviews consistently single it out for design and ease of use, especially the iOS app — it feels like a native Apple utility, not a web app in a wrapper. Job cards are white, flat, and carry one accent colour for status and nothing else. The web admin is a thin dashboard; the phone is the product.
- **Worth stealing.** The discipline of one status colour per card and nothing else coloured. Kirei's own leads redesign already arrived at this (3px stage rail on a neutral card) — ServiceM8 is proof it's the right call for this exact audience. Also: their job card leads with the customer name and address, not a reference number. Kirei's invoice row leads with a gradient icon tile that carries no information.

Tradify

- **Does well.** Reputation is built on 'simplicity, ease of use and fast setup'. The UI is deliberately boring — grey/white, one blue accent, tables that look like tables. A plumber can be productive in an afternoon.
- **Worth stealing.** Boring is a feature for this audience. Tradify never asks the user to decode a colour. Kirei currently hashes customer names into nine saturated gradients (rose, coral, amber, violet, dusk, ocean) — a tradie sees a rainbow and correctly assumes the colours mean something. They don't.

Jobber / Housecall Pro

- **Does well.** Drag-and-drop scheduling calendar as the centre of gravity, with real-time state. Both use a restrained neutral canvas with a single brand green/blue, and reserve saturated colour strictly for job status on the calendar. Jobber is described as the more intuitive of the two for new users.
- **Worth stealing.** Colour budget: their calendars can afford saturated status blocks because every other surface is neutral. Kirei spends its colour budget on decorative gradient tiles in list rows and quick-action buttons, so when it genuinely needs to shout (overdue invoice, unassigned job) it has nothing left.

simPRO / AroFlo / Fergus

- **Does well.** simPRO is the density benchmark — real data tables, tabular numerals, sortable columns, sticky headers, built for an office admin who lives in it eight hours a day. Fergus sits mid-market with strong quoting screens.
- **Worth stealing.** simPRO is what Kirei's `.ch-table` already is and should stay. The mistake would be replacing tables with card lists everywhere — Kirei has done exactly that on `/invoices`, `/quotes`, `/customers` while keeping real tables on `/expenses`, `/inventory`, `/assets`, `/timesheets`. Pick one per data shape and be consistent.

Linear

- **Does well.** Near-monochrome. Colour appears only as small semantic dots and priority icons. Extremely tight vertical rhythm, keyboard-first, and every row is the same height so the eye scans a column, not a mosaic. Load states are skeletons that match final layout exactly, so nothing reflows.
- **Worth stealing.** Fixed row height and a single-column scan target. Kirei's invoice rows use `flex-wrap` with `min-w-[140px] basis-0` (`invoices-client.tsx:186`), so rows reflow to different heights at intermediate widths and the scan breaks.

Stripe Dashboard

- **Does well.** One indigo accent, ~22 total colours in the whole system, and heavy use of tabular numerals with right-aligned money. Status is a small quiet pill; the number is the loud element. Restraint is the entire design thesis — the gradient does the heavy lifting on marketing pages, and the product surface is almost colourless.
- **Worth stealing.** Money should be the biggest, boldest thing in an invoice row — Kirei renders the total at `text-sm font-semibold` (`invoices-client.tsx:198`), the same visual weight as the customer name, while a 40px saturated gradient tile sits to the left carrying zero information. Invert that hierarchy.

Recommendations

[HIGH] Colour system

- **Problem.** Four independent status-colour vocabularies are in simultaneous use: `.ch-pill` (`globals.css:455-493`), `KireiPill` (`ui/kirei/pill.tsx:44-51`), the leads `STAGES` map (`leads/lead-shared.ts:23-29`), and `STATUS_GRADIENT` icon tiles (`invoices-client.tsx:183`). The same status renders three different ways depending on which page you are on, so the user can never learn one visual language.

- **Change.** Make `KireiPill` the single status primitive and delete the other three. Reimplement its `TONE_CLASS` map against the Connected Hub tokens rather than raw Tailwind palettes — i.e. `success` becomes the warm mint already defined for `.ch-pill.paid`, not `bg-emerald-100`. Keep the leading dot from `.ch-pill::before` (it aids colour-blind users) and drop the ALL-CAPS 10px styling in favour of `.ch-pill`'s 11.5px sentence case, which is more legible on a phone in sunlight. Then rewrite `.ch-pill` in `globals.css` as a thin alias that shares the same custom properties, so the 6 files still using the class inherit the fix for free. Leads keeps its stage rail (that part is correct) but sources the rail colour from the same token set.
- **Where.** `src/components/ui/kirei/pill.tsx` (rewrite `TONE_CLASS`); `src/app/globals.css:455-493` (re-point `.ch-pill` at shared vars); `src/components/leads/lead-shared.ts:23-29` (`STAGES` soft/dot derive from tokens); `src/components/invoices/invoices-client.tsx:183` (drop `STATUS_GRADIENT`)

[HIGH] Dark mode correctness

- **Problem.** `.ch-pill` has no dark-mode variant anywhere in `globals.css` — backgrounds are hardcoded literals like `hsl(150 50% 95%)`. In dark mode every one of these pills renders as a near-white block. It ships on the worker dashboard, the messages list and the briefing bell. `.ch-stat-delta.up` / `.down` (`globals.css:412-413`) have the identical defect: dark green/red text on a dark card.
- **Change.** Convert every hardcoded `hsl(...)` literal in the `.ch-pill` and `.ch-stat-delta` blocks to a pair of CSS custom properties defined once at `[data-theme="console"]` and overridden in the existing `.dark [data-theme="console"]` block (`globals.css:158-179`). In dark mode the pattern should be a ~20% alpha tint of the hue with a light foreground, mirroring what `KireiPill`'s `dark:bg-emerald-900/40` already does correctly. This is a genuine rendering bug on the crew-facing screen, which is the surface most likely to be used outdoors in dark mode.
- **Where.** `src/app/globals.css:412-413` and `474-493` (add dark overrides in the block at 158-179)

[HIGH] Rendering / theme flash

- **Problem.** `useIsDark()` is duplicated verbatim in `gradient-tile.tsx:25-37`, `stat-tile.tsx:25-37` and `avatar.tsx:26-38`, and each copy initialises to `false`, only correcting inside `useEffect`. SSR and first client paint therefore always emit the LIGHT gradient palette. In dark mode every `GradientTile`, `StatTile` and `KireiAvatar` in the app visibly flashes pale teal before flipping — on the dashboard that is a dozen elements strobing on every load.
- **Change.** Delete all three `useIsDark` copies and render the already-written-but-unused `gradientVars()` helper at `gradient-tokens.ts:63-74`, which emits `--g-from` / `--g-to` / `--g-from-dark` / `--g-to-dark` as inline custom properties. Add one rule in `globals.css` under the dark selector that remaps `--g-from: var(--g-from-dark)`. The gradient then resolves entirely in CSS with correct values on the server render, no JS, no `MutationObserver`, no flash — and the three components become server-component-safe, removing three `"use client"` boundaries.
- **Where.** `src/components/ui/kirei/gradient-tile.tsx`, `stat-tile.tsx`, `avatar.tsx` (remove hook, consume `gradientVars`); `src/app/globals.css` (add `--g-*` dark remap); `gradient-tokens.ts:63-74` (already written)

[HIGH] Colour restraint

- **Problem.** `KireiAvatar` hashes every customer/lead name into a nine-gradient pool (`avatar.tsx:12`) drawn from fully saturated tokens — rose `#fb7185`→`#9f1239`, coral, amber, violet, dusk, ocean (`gradient-tokens.ts:12-29`). A 50-row customer list is a rainbow in which colour encodes nothing, training the user to ignore colour precisely where the app later needs it to mean something (overdue, unassigned, failed payment). The dashboard compounds this with four different saturated gradients on four equally-weighted quick actions (`dashboard-client.tsx:150-153`).

- **Change.** Reduce the avatar `POOL` to a monochrome ramp: 4-5 tints of the teal primary at differing lightness, so avatars still disambiguate adjacent rows but read as one family. Alternatively drop the gradient entirely for a flat `hsl(var(--muted))` chip with `hsl(var(--foreground))` initials, which is what Linear and Stripe do. Separately, set all four dashboard quick actions to `gradient="primary"` — they are four equal actions and colour-coding them implies a taxonomy that does not exist. Reserve the saturated `emerald` / `rose` / `amber` gradients exclusively for genuine semantic states.
- **Where.** `src/components/ui/kirei/avatar.tsx:12` (`POOL`); `src/components/dashboard/dashboard-client.tsx:150-153` (`HubTile` gradients)

[HIGH] Visual hierarchy — money

- **Problem.** In the invoice list row (`invoices-client.tsx:170-200`) the total renders at `text-sm font-semibold` — identical weight to the customer name — while a 40px saturated gradient tile carrying no information occupies the leading position. The single field the business owner actually scans for is tied for third in visual priority behind a decorative square.
- **Change.** Invert it. Drop the gradient icon tile from the row entirely (status is already communicated by the pill; the tile is redundant encoding). Render the total at `text-base font-bold tabular-nums` right-aligned, and demote the invoice number to the existing muted mono treatment. This mirrors Stripe: the number is the loud element, status is a quiet pill. Apply the same inversion to quotes. The row also gains vertical space back, letting more invoices fit above the fold — density that matters to the one real user carrying \$17k outstanding and \$327k in open quotes.
- **Where.** `src/components/invoices/invoices-client.tsx:183-198`; mirror in `src/components/quotes/quotes-client.tsx`

[MEDIUM] Data display consistency

- **Problem.** Two answers to the same question coexist. Seven views (`expenses-view`, `inventory-view`, `assets-view`, `timesheets-view`, `prospects-view`, `analytics-client`, `seo-performance`) use the real `.ch-table` with tabular numerals, mono refs and uppercase headers. `Invoices/quotes/customers/work-orders` instead hand-roll a table out of divs with fixed-width spans (`w-24`, `w-28`, `w-32`) and `divide-y` — forfeiting the numeric alignment, semantics and keyboard behaviour, and using `flex-wrap` so rows reflow to ragged heights at tablet widths and column scanning breaks.
- **Change.** Pick per data shape and enforce it: money/date/reference records render as `.ch-table` on desktop; only genuinely card-shaped data (leads on a board, photo-driven job cards) stays as cards. Convert the `invoices/quotes` list to `.ch-table` at `md:` and up, keeping the current stacked card layout below `md:` via a single responsive switch. This immediately buys right-aligned tabular-nums money columns (`.ch-table .num`) and fixed row heights, and removes the flex-wrap reflow.
- **Where.** `src/components/invoices/invoices-client.tsx:170-200`; `src/components/quotes/quotes-client.tsx`; `src/components/customers/customers-client.tsx`; `src/components/work-orders/work-orders-client.tsx`

[MEDIUM] KPI strip consistency

- **Problem.** Two competing KPI treatments ship simultaneously: `.ch-stat` (`globals.css:379-411`) is a plain white card and is used on `/analytics` and the worker dashboard; `StatTile` (`ui/kirei/stat-tile.tsx`) is a tinted-gradient card used across eleven other surfaces. Additionally `StatTile` hardcodes `rgba(255,255,255,0.55)` for its icon chip (line 53), producing a glary white chip on the `#143536` dark-mode tile, and the light-mode tone colours are passed as raw hex strings at every call site (`invoices-client.tsx:131-140`).

- **Change.** Standardise on one. Recommend keeping `StatTile`'s structure but dropping the gradient background for a plain `bg-card border-border` surface with the tone colour surviving only in the small icon chip — this converges the two treatments, matches the `.ch-stat` that already exists, and removes a second place saturated colour leaks in. Replace the hardcoded white chip with `hsl(var(--card))` at reduced alpha so it adapts. Replace the raw hex `toneColor` props with a named tone union (`"primary" | "success" | "warn" | "danger"`) resolved inside the component, so call sites stop hardcoding `#1f4f4a` and `#9f1239`.
- **Where.** `src/components/ui/kirei/stat-tile.tsx:39-67` (surface + chip + tone prop); the 8 call sites listed by grep for `StatTile`; `src/app/globals.css:379-411` (retire `.ch-stat` or alias it)

[MEDIUM] Empty states

- **Problem.** Two empty-state treatments exist: `.ch-empty` (`globals.css:716-722`) is a plain centred text block, while `EmptyState` (`ui/kirei/empty-state.tsx`) renders a 64px saturated gradient halo plus CTA. Neither is wrong, but the gradient version is the louder element on an otherwise-empty page — a brand-new customer's very first impression of every screen is a large saturated blob. For an app with 7 businesses and heavy onboarding pressure, empty states are disproportionately load-bearing.
- **Change.** Consolidate on `EmptyState` and retire `.ch-empty`. Soften the halo: use the `softTeal` gradient rather than the saturated `primary` default (`empty-state.tsx:20`), which keeps the shape without the shout. More importantly, make the copy do the work — every empty state should name the next concrete action in the user's language ("No invoices yet — bill your first job") and always supply the `cta`, which is currently optional and therefore frequently omitted. A first-run user staring at eight empty screens with no CTA is the single most expensive design failure for conversion.
- **Where.** `src/components/ui/kirei/empty-state.tsx:20` (default gradient, make cta required); `src/app/globals.css:716-722` (delete `.ch-empty`); audit all `EmptyState` call sites for missing cta

[MEDIUM] Brand consistency

- **Problem.** `src/app/layout.tsx:9` sets `themeColor: "#2563eb"` — bright blue from the abandoned `:root` design system — while the entire app runs `[data-theme="console"]` with a deep teal primary `191 38% 36%`. On an installed PWA and on Android, the browser chrome and status bar therefore render blue. That is the first colour a customer sees on their phone, and it belongs to a theme the product no longer uses.
- **Change.** Set `themeColor` to the resolved deep teal (`#38797a`, the hex of `hsl(191 38% 36%)`), or better, supply the light/dark pair via the `media` variant so it tracks the canvas — `#fafaf7` for light and `#141413` for dark, which is what most polished PWAs do so the chrome disappears into the page. While in there, the dead `:root` blue token block (`globals.css:40-71`, `--radius: 0.5rem`) is fully shadowed by the console theme on every route and can be deleted, removing a second radius scale that contradicts the one CLAUDE.md documents.
- **Where.** `src/app/layout.tsx:9` (`viewport.themeColor`); `src/app/globals.css:40-102` (retire the dead `:root/.dark` blue theme)

[LOW] Token discipline

- **Problem.** The `.ch-*` utilities hardcode `border-radius: 14px` in `.ch-stat` (`globals.css:382`) and `.ch-table-wrap` (`globals.css:420`) rather than referencing `var(--radius-xl)`. CLAUDE.md documents `--radius: 0.625rem` as the single source of truth resolving rounded-md/lg/xl to 8/10/14px — these two rules coincidentally match today, so any future radius change will silently desync the KPI strip and every table from the rest of the app. The same class of drift appears in `.ch-quick-action` (10px) and `.ch-activity-icon` (8px).

- **Change.** Replace every hardcoded px radius in the `.ch-*` block with `var(--radius-xl)` / `var(--radius-lg)` / `var(--radius-md)` as appropriate. Mechanical, zero visual change today, and it makes the radius token actually authoritative as CLAUDE.md claims it is.
- **Where.** `src/app/globals.css:382, 420, 651, 671, 692` (and any other literal px radius in the `.ch-*` section)

[LOW] Motion

- **Problem.** Interaction feedback is inconsistent in kind, not just degree. Leads rows use `hover:border-primary/30` (`leads-list.tsx:96`), invoice rows use `hover:bg-muted/40` (`invoices-client.tsx:182`), `.ch-table tbody tr:hover` uses `background: hsl(var(--muted) / 0.6)` at 0.08s (`globals.css:446-450`), and `.ch-quick-action` swaps both background and border at 0.12s. Four different hover languages across four surfaces the same user touches in one session.
 - **Change.** Standardise on one row-hover treatment — recommend `.ch-table` 's `hsl(var(--muted) / 0.6)` at 80ms, which is the fastest and least distracting — and apply it to the card-list rows too, dropping the border-colour hover. Reserve border-colour change exclusively for genuinely selectable/draggable surfaces (lead cards on the board), where it correctly signals affordance rather than mere hover.
 - **Where.** `src/components/leads/leads-list.tsx:96`; `src/components/invoices/invoices-client.tsx:182`; `src/components/ui/kirei/card-list-row.tsx:26`; `src/components/ui/kirei/hub-tile.tsx:28`
-

Appendix D — Findings rejected in verification

Raised by an audit agent, then **refuted** when checked against the real code. Listed so the reasoning is preserved — one of these would have broken production if actioned.

The invoice/parent payment rollup is implemented twice, differs between the two copies, and neither copy is tested

REFUTED on its central claim. The finding says the two copies "already differ" because `recomputeParent` preserves cancelled/draft while `addPayment` "unconditionally overwrites status (invoices.ts:370)". That comparison is apples-to-oranges: `invoices.ts:370` is the `recompute` of the invoice ITSELF, whose counterpart is `recomputeInvoice` (`stripe-payments.ts:186`) — and that one is also unconditional. I read `invoices.ts:400-420`, which is the actual parent branch, and it is behaviourally identical to `recomputeParent`, including the same guard verbatim: `if (currentStatus !== "cancelled" && currentStatus !== "draft")` at `invoices.ts:412`, same 0.01 tolerance, same `amount_paid` formula. The two implementations agree. What IS true: the logic is duplicated in two places and no test file imports either module. But the asserted divergence — the whole basis for the 'high' severity and the 'cancelled invoice flips back to partial' impact — does not exist in the code.

The pragmatic minimum suite before charging customers: five files, roughly two days

Not a finding — this is a work plan assembled from the other eight items, so there is nothing to confirm or refute as a defect. Its underlying facts do check out (the cited files and line ranges are accurate, and I independently confirmed the zero-coverage claims on `stripe.ts`, `payment-methods.ts` and the rollup). But item 2 of its plan rests on the payment-rollup 'divergence' I refuted above, and it should not carry a severity or appear in a findings list. Fold its recommendations into the individual findings instead.

CLAUDE.md documents /api/agent as both live and retired; the 2,151-line "dead" route is still in the tree

The documentation contradiction is real (`CLAUDE.md:181` describes `/api/agent` as the live floating panel with 65 tools; `CLAUDE.md:264` calls it "the old `/api/agent`" with 79 tools). But the finding's load-bearing claim — that the route is dead and should be deleted — is REFUTED. `src/components/agent/agent-panel.tsx:246` still does `fetch("/api/agent", ...)`, and `src/components/layout/dashboard-shell.tsx:13-14,82` dynamically imports and renders `<AgentPanel />` on every dashboard page. Both routes are live: `/api/assistant` backs the `/assistant` page, `/api/agent` backs the floating panel. Following this finding's recommendation ("Delete `src/app/api/agent/route.ts`") would break the in-app floating assistant in production. Task #73 "retire `/api/agent`" is indeed marked completed while the route still serves traffic — that is the real, narrower issue, but it is a task-tracking/architecture gap, not the dead code the finding describes.